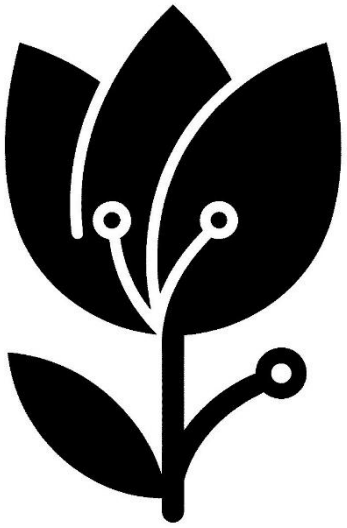
TULIP4041 – The ULtimate Intelligent Peripheral for the 41



Introduction

In the mid 00's I had finished the MLDL2000 project, and started to consider a follow-up project that would be named the MLDL3000. Most important addition would be a microcontroller to perform more complex tasks, such as interfacing to a PC, printer emulation and saving and storing user programs and ROM images. This resulted in a working prototype, but real life and family interfered. And with the introduction of Diego's Clonix and NoVRAM modules and finally Monte's HP41CL I thought that the MLDL2000 or -3000 would simply be superfluous.

After starting to reduce my working hours I picked up some activities again in 2018, and late 2022 I decided to give it a go again, now with a slightly different platform. But still using an FPGA for the HP41 bus interfacing, and a microcontroller for the high-level stuff. In the spring of 2023 Andrew Menahue posted a message and video on the HP Museum forum, and that did it. A new approach using the RP2040 microcontroller that did all the work, including the low-level bus interfacing. Not entirely new, as Diego's modules also used a microcontroller to interface with the HP41 bus. The RP2040 however has so much more performance that it could run more complex tasks. After studying the datasheet I made a decision to go for it and in the summer of 2023 a first breadboard version was working. My initial goal was to use the PIO in the RP2040 for the HP41 bus interfacing, and to create emulation of the HP-IL module. After that steps were taken to create a product that is useable for the community.

During October 2024 the first units of the DevBoard were shipped and design of the module version started, and the design has been migrated to the RP2350 processor. After designing the final PCB and housing (by Andrew Menahue) and two design iterations In October 2025 the first final units were shipped.

This document serves as the User Manual. It is strongly recommended to read the TULIP Background document to get an understanding of how it works. If you want a quick start with your TULIP then start reading with Chapter 11 but reading the complete manual will help you get the most out of it

Meindert Kuipers *Email: meindert@kuiprs.nl*

November 2025

1. Credits

Many thanks to Andrew Menahue for inspiring this project, his cooperation and the design of the TULIP housing, Thomas Fänge for his contributions, cooperation and testing. HP-IL emulation and MOD file handling ing is based on V41 and EMU41 by Christoph Giesselink and Jean-Francois Garnier, PILBox emulation is based on the PILBox from Jean Francois Garnier.

2. Version info

The TULIP4041 is a project that is in constant change. This documentation may not match the current available version but rather serves as a specification for the TULIP4041, and this means that is may describe functionality that is not (yet) implemented. Refer to the TULIP help menu for the actual supported functions of your firmware version.

IMPORTANT: This document and firmware is migrated to the RP2350. The RP2040 is no longer supported by the TULIP firmware.

3. Conventions

`00FF`	Hexadecimal numbers are used to indicate addresses, leading zeroes are typically used to indicate the total possible range, e.g. 00FFFF. The 'x' character is used for a "don't care" situation
`0x040`	The 0x sequence is used to indicate hexadecimal values when context is not clear
`0b1010`	Binary values are preceded with the 0b sequence, the 'x' character is used for don't care bits
HP41	HP41 indicates all versions of the HP41 calculator, including HP41C, CV, CX, CL, DM41X
41CL	Is used when specifically referring to the HP41CL system, also 41CL is used
help	is a command to be typed in the TULIP4041 Command Line Interface (CLI)

4. Copyrights and Disclaimer

Information in this document and the function of the hard- and software has been carefully checked and is believed to be accurate as of the date of publication; however, no responsibility is assumed for inaccuracies. I will not be liable for any consequential or incidental damages arising from reliance on the accuracy of this document and the use of the hard- and software. The information contained herein is subject to change without notice.

Copyright © 2026 Meindert Kuipers. This document and all sources and schematics are subject to the MIT license conditions.

Table of Contents

Introduction 1
1. Credits 1
2. Version info 2
3. Conventions 2
4. Copyrights and Disclaimer 2
Table of Contents 3
5. TULIP4041 overview 5
6. TULIP4041 interfaces 8
6.1. TULIP4041 memory overview 8
7. TULIP4041 power consumption 9
8. TULIP4041 Firmware programming or update 10
9. Using the TULIP4041 11
9.1. Virtual serial ports 11
9.2. Reset of the TULIP4041 12
9.3. Signalling LED 12
9.4. Micro SD card 12
10. ROM images, User memory and the File System 14
10.1. Importing files and the File Manager 15
10.2. Exporting files 18
10.3. Files containing QROM 18
10.4. Plugging ROM images 19
10.5. Bank switching in the TULIP 21
10.6. Embedded ROM images 22
10.7. Hardware emulation 23
10.8. Unplugging ROM images 23
HP41 User Memory 24
10.9. Using the mcode tracer 26
10.10. HP-IL emulation 28
10.11. HP82143A Printer Emulation 29
10.12. Using HEPAX in the TULIP4041 30
10.13. The HP82153A Barcode Reader (Wand) 31

11. TULIP4041 first steps 34
12. TULIP4041 Command Line Interface reference 35
15. Limitations 51
16. TULIP4041 Hardware: Module version 52
16.1. The TULIP housing 52
16.2. Soldering the connector 52
16.3. Installing the battery board 56
16.3.1. Installing the battery board on V1.0 hardware 56
16.3.2. Installing the battery board on V1.1 hardware 57
16.4. Preparing for first use 58
16.5. The micro SD card 59
16.6. Other tests 60
16.7. TULIP Module IO connections 60
17. TULIP4041 Hardware: Development Board 63
17.1. Assembling the DevBoard 63
17.2. Prepare the micro SD card 65
17.3. Getting the DevBoard up and running 65
18. References 70
19. Change log 72

5. TULIP4041 overview

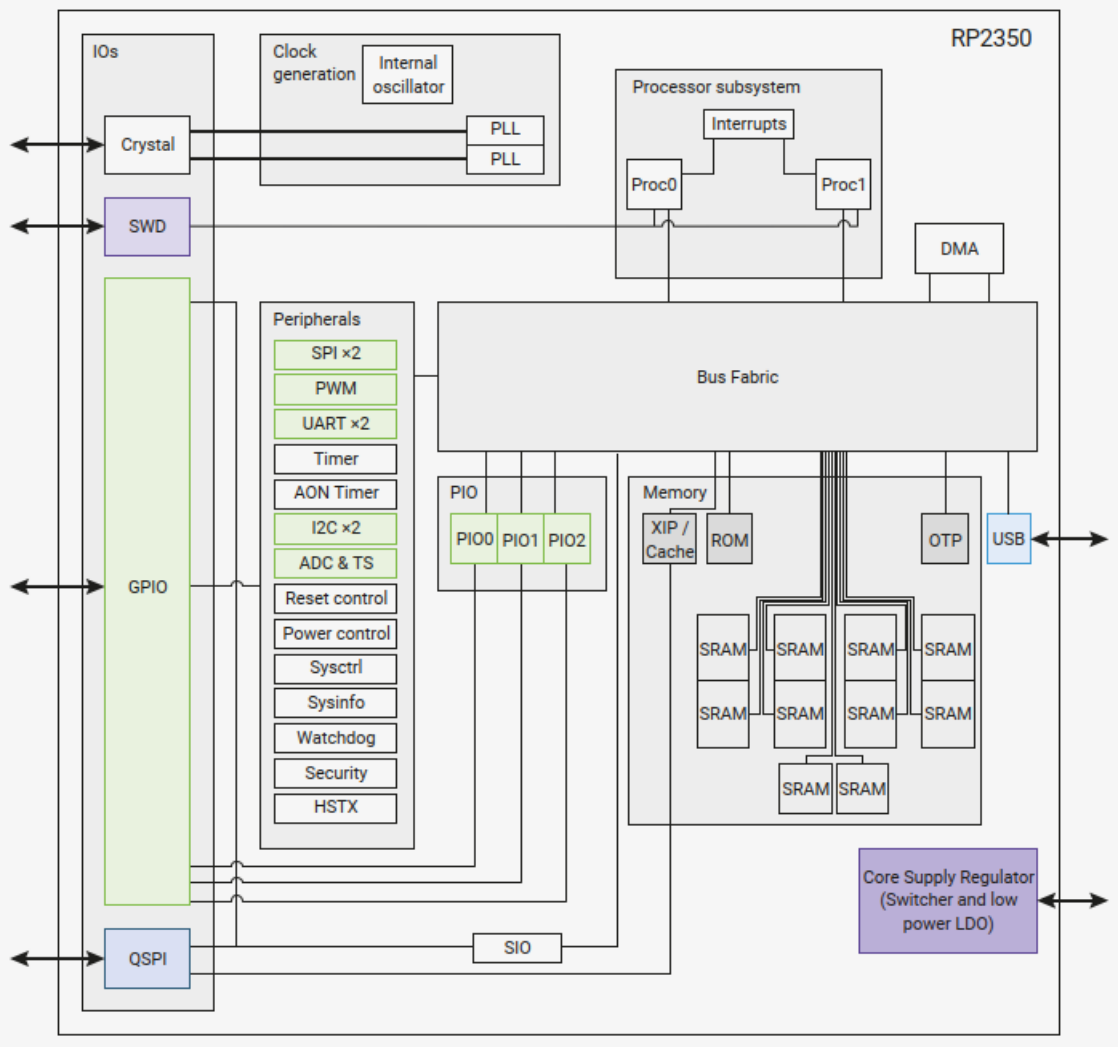
The TULIP4041 is a smart peripheral for the HP41 series of calculators. It is capable of emulating nearly all of the original memory and peripheral modules and offers a number of new features for the HP41. Some of the features it supports:

- Extended Functions and Extended Memory module emulation
- User Memory emulation
- HP-IL and HP-IL printer to virtual devices on a host computer
- HP82143A printer by printing to an IR port or virtual serial port to a printer simulator on a host computer
- Plugging virtual ROM and MOD images, including those with MLDL (QROM) functionality
- HEPAX emulation
- TIME module emulation (in development)
- USB-C interface
- Easy firmware upgrade
- Open Source hardware and firmware (MIT license)
- Micro SD card that appears as a USB thumb drive on the host computer for exchanging ROM and MOD images
- Multi-port virtual serial interface to the host computer for the following functions:
    - Command Line Interface for plugging/unplugging ROMs and control functions
    - HP41 mcode level bus tracer with disassembler
    - Virtual HP-IL serial interface according to the PILBox protocol for connection with virtual HP-IL devices on the host computer
    - HP-IL frame monitoring
    - Virtual printer connection for use with a HP82143 simulated printer of a host computer
    - Most functions are platform independent and work on Windows, Apple and/or Linux

Note: many of the above functions are implemented, some are not (yet) implemented.

The TULIP4041 firmware is originally designed for the RP2040 microcontroller and has been migrated to the RP2350A in September 2024. This controller is designed by Raspberry Pi. The device is very low cost (typically between €1 and €2 depending on quantities) and offers the following main features:

- Dual-core Arm Cortex-M33 processor, flexible clock running up to 150 MHz (overclocking possible)
- 520 kByte on-chip SRAM
- Up to 16 MByte external QSPI flash
- 2 × UART, 2 × SPI controllers, 2 × I2C controllers, 24 × PWM channels
- 1 × USB 1.1 controller and PHY, with host and device support
- 12 × Programmable I/O (PIO) state machines (3 PIO blocks) for custom peripheral support
- Operating temperature -40°C to +85°C
- Drag-and-drop firmware programming using mass storage over USB
- Low-power sleep and dormant modes
- Temperature sensor
- Accelerated integer and floating-point libraries on-chip
- Excellent support for multi-platform development tools and many (open source) libraries



RP2350 BLOCK DIAGRAM (FROM RP2350 DATASHEET)

The TULIP4041 uses additional hardware:

- 4 or 16 MByte of FLASH memory, part of the RP2350 structure, code actually runs from the flash (XIP) and most of the ROM images are stored in flash and directly accessed there (4 MByte for the DevBoard with the standard Pico2 module and 16 MByte on the module version)
- 256 KByte FRAM memory to emulate QROM, user memory and for saving system settings
- Micro SD card holder
- Infrared LED for IR printer operation
- Level shifters to interface with the HP41 bus
- Real Time Clock (on the module version only)

RP2350 development boards are available from several vendors, including the Raspberry Pico2 which is used in the prototyping phase of this project.

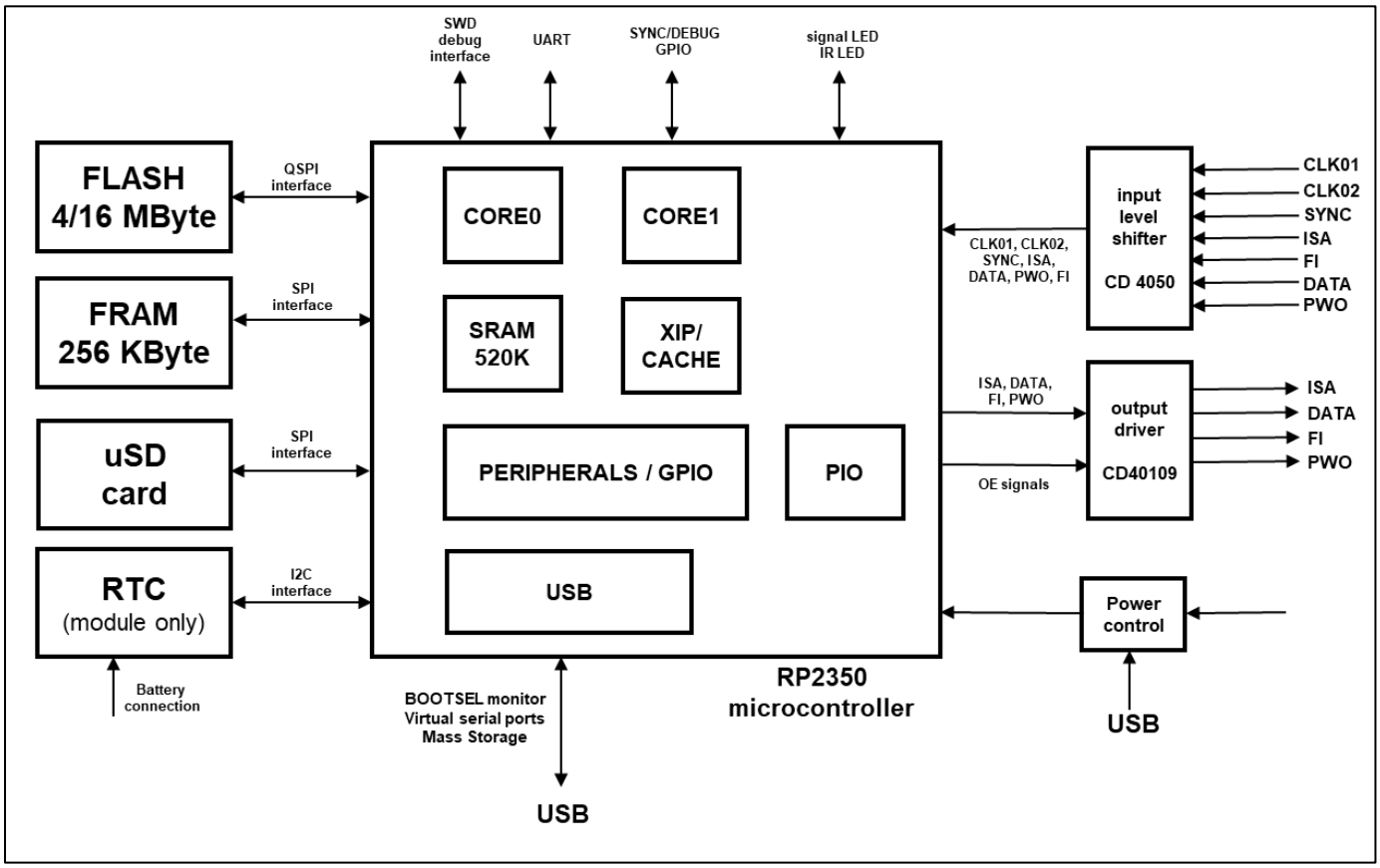
In this project both ARM cores are used, and two of the PIO blocks are used to almost their full capacity for the HP41 bus interfacing. The RP2350 has two RISC-V Hazard3 processor cores that can be enabled instead of the ARM cores, these are not used on the TULIP4041.

The TULIP4041 comes in two different versions:

- TULIP DevBoard: a hand solderable board with all interfaces available on headers for development and debugging. This version is mainly intended for my own development to replace the fragile breadboard. It uses a standard Pico2 board (or 100% pin compatible) and comes with a connector board for plugging in the HP41
- TULIP Module a long module sized PCB with all smd components assembled except the HP41 module connector. This unit has the size of an HP-IL module and is housed in a custom 3D printed module housing

Both versions have their own specific firmware, but the functionality is nearly identical. The versions are derived from a single set of sources with conditional compiles.

IMPORTANT: the HP41 module connector is NOT included with either product!



TULIP4041 Block diagram

6. TULIP4041 interfaces

The TULIP4041 has a number of interfaces to the outside world:

- HP41 module interface, using level shifters to convert the 6V HP41 signals to the 3.3 V RP2350 compatible levels, and vice versa.
- RP2350 debug interface
- RP2350 USB interface
    - In BOOTSEL mode it offers the possibility to upload a new firmware image and to program FLASH memory
    - In normal running mode (TULIP4041 firmware) offers the following virtual serial ports (CDC)
        - Command Line Interface (CLI)
        - HP41 mcode/bus trace output
        - HP-IL communication
        - HP-IL scope output
        - Printer data (output from TULIP4041 only)
        - Mass Storage (MSC) device for exposing the micro SD card to the host computer
- Micro SD card
- Real Time Clock (only on the Module version) for possible TIME module emulation
- Serial interface (digital 3.3V level only)
- RP2350 GPIO pins
- IR led
- On-board signalling LED
- Auxiliary signals for multi-core communication, debugging en future additional functions
- I2C signals for connecting I2C peripherals (used for the RTC) like an external display

The TULIP4041 interfaces with the HP41 using level shifters. These are needed to convert the 6V HP41 signals to the 3.3 V RP2350 compatible levels, and vice versa. All other signals are 3.3V (please check the RP2350 datasheet for details).

6.1. TULIP4041 memory overview

The TULIP4041 has a number of different memory types

- 520 KByte on-chip SRAM, used by the RP2350 for its own data and execution of critical parts of the program. This memory is dual ported for fast access by both cores. When used, the TraceBuffer typically takes up most of the available SRAM. Biggest user of RAM is the queue used for the tracebuffer. In a future firmware version it is anticipated that a copy of emulated User Memory will reside in RAM.
- 4 MByte FLASH memory (on the Pico2 board for use with the Development Board), used for program storage (and program execution) and storage of ROM images. The first 1 MByte is reserved for code storage, the rest is available for ROM images
- 16 MByte FLASH memory on the module version, of which the first 1 Mbyte is used for firmware code storage, leaving 15 MByte for the file system
- 256 KByte FRAM, used for QROM images, Extended/Expanded/User memory emulation and persistent storage of settings
- Micro SD card, storage used for ROM images, user programs (RAW files) and LIF container for HP-IL drive simulation.

All storage is managed by the firmware under control of the User Interface (CLI). The SD card must be formatted by the user in a host system with the FAT or exFAT file system, and the subdirectories with ROM images and RAW files must be created in that system as well.

7. TULIP4041 power consumption

The HP41 system is generally a very low power system. The RP2350 however is a very performant and somewhat power hungry processor. Care should be taken to properly manage power consumption in combination with the HP41. The TULIP manages power consumption by reducing the CPU clock speed when it is not powered by USB.

USB power connected:
This is detected with the VBUS input (GPIO24 on the processor). CPU speed is then 150MHz, an the Peripheral Clock is also 150MHz. The power consumption is 23-24mA

Powered by the HP41:
In this case VBUS (derived from the USB power) is low and the firmware switches the CPU clock to 75MHz. This reduces the power consumption to about 15 mA (excluding the HP41 itself). The peripheral clock remains at 150MHz to ensure that the clock frequency of especially the FRAM and UARTS are not affected.

Currently the RP2350 processor is always running when the unit is powered.

A microSD card will consume around 1-2 mA when idle, and up to 20-30 mA extra when in use. Some high-speed cards consume even more, and it is advised to check power consumption of the cards. Programming (writing to) FLASH memory increases power consumption with about 10mA. During power up of the TULIP the uSD card can sometimes use op to 20-30 mA extra while being mounted (additional to the normal power consumption).

[NOT YET SUPPORTED]: When not powered by USB the TULIP4041 enters low power mode after about 10 seconds when the HP41 goes into STANDBY (also referred to as LIGHT SLEEP) or OFF (DEEP SLEEP) mode The power consumption is then still around 2.7 – 3 mA, and this will drain the HP41 battery quickly. The TULIP4041 is powered by the HP41 at all times when not powered by USB. Power is sourced by the HP41 battery directly and not by the HP41 regulator. The processor will NOT power down when one of the virtual serial ports is active or during file operations or FLASH/FRAM programming. When a USB power bank is connected, no serial ports are active and the power bank will power the TULIP4041 in low power mode. Please be aware that many USB power banks require a minimum current to prevent them from being shut down and that the low current of the TULIP may cause the power bank to shut off power.

When powered by USB the calculator is always powered by its own battery.

Advice 1: Connect the TULIP4041 to a USB power source whenever possible and practical

Advice 2: Remove the TULIP4041 from your calculator when not in use and when it is not connected to a USB power source.

When powering down the TULIP4041 will save all relevant settings and memory contents to non-volatile storage. These contents are then safe when the TULIP4041 is removed from the calculator. Take special care when User Memory or Extended Memory is active. Powering the HP41 with the TULIP with these settings may lead to a MEMORY LOST. Also do not move the TULIP to another calculator that does have a Quad or more Extended Memory already installed.

8. TULIP4041 Firmware programming or update

Updating the TULIP Firmware is relatively easy, just follow the steps below. When using the DevBoard and you are using a new Pico2 this board will always start up in the so-called BOOTSEL mode. When connecting to a host compute in BOOTSEL mode you will see a drive with the name RP2350. Simply copy the firmware file (the file with extension .uf2) to this drive and the TULIP processor will restart with the new firmware.

BE CAREFUL: The DevBoard and the Module use different firmware files!

ADVICE: Do the firmware upgrade with your HP41 disconnected!

OOPS: You may lose some of your TULIP settings (although unlikely) after a firmware upgrade

UNPLUG: all plugged Embedded ROM's (HP-IL, Printer) MUST be unplugged prior to a firmware upgrade. This is automatic starting with firmware 0.95

To enter BOOTSEL mode on the DevBoard:

- If no previous TULIP firmware was loaded you may use the PicoTool, or push the little button while plugging the USB cable. Please refer to the Pico documentation for more information.
- If there is TULIP firmware loaded simply use the CLI command ***system BOOTSEL***

To enter BOOTSEL mode on the TULIP Module:

- Use the CLI to issue the ***system BOOTSEL*** command, this is the preferred method. This will work only if there was already a TULIP firmware running. This command unplugs all embedded roms (from firmware 0.95). Sometimes the RP2350 drive is not exposed after the ***system BOOTSEL*** command. Simply power cycle the TULIP (by removing USB power) and try again.
- In rare cases when the firmware is locked a bit of fiddling is required. Power cycle the TULIP (disconnect from the calculator) and try again to access the CLI. If that does not work remove the TULIP from the calculator and disconnect the USB cable. Open the TULIP housing and locate the USB_BOOT pad (pin 6 on the large set of connector pads). Use a short piece or wire to connect this pad with GROUND (the square pad 7, next to pad 6). Now connect the USB cable again while keeping USB_BOOT and GROUND shorted. Release the connection between USB_BOOT and GROUND when the USB connection is made and the TULIP will be in BOOTSEL mode
- With a development system and Pico Probe connected through the debug interface new firmware can be downloaded using the development tool (VS Code or OpenOCD)

Once the TULIP4041 firmware is loaded it is not possible to use Picotool to get the processor in BOOTSEL mode.

The RP2350 processor uses BOOTSEL mode for a number of internal functions such as firmware update, FLASH programming and other settings. Also the update of the TULIP4041 firmware requires entering BOOTSEL mode. In this mode the processor enables a special USB interface (for FLASH programming for example). Please refer to the RP2350 documentation and tools for a complete description.

IMPORTANT: due to the use of multiple virtual serial ports it is not possible to enter BOOTSEL mode under control of the PICOTOOL utility on the host computer when the TULIP4041 firmware is running. Once in BOOTSEL mode the various functions of PICOTOOL can be used.

9. Using the TULIP4041

The primary interface for using the TULIP4041 is the USB interface. This offers a number of virtual serial ports (VCP) to the host system and a USB storage medium. On a Windows system these will be visible as COM ports, on Linux these will typically be /dev/ttyACMx ports). The next section assumes a Windows host computer. Due to the handling of these ports on a host system it can sometimes be tricky to identify a port, and this can change as well. The assignment of the COM port numbers appears to be somewhat arbitrary. On a Windows system, use the Device Manager -> Communication Ports to get a list of which COM ports are used. The storage medium is the micro SD card and a drive letter will be assigned by the host computer, even when no micro SD card is plugged on the TULIP.

When the TULIP4041 is connected (via USB) to a host computer the first action is typically to connect to the CLI with a terminal program.

Normally the TULIP4041 is plugged in an HP41 calculator for use. It is very well possible to connect a unit to USB without being plugged in an HP41.

The virtual devices will be visible only when valid firmware is loaded and running on the TULIP4041. Please refer to the section in one of the next paragraphs on instructions to load the TULIP firmware.

9.1. Virtual serial ports

All COM ports use 8 bits, 1 stop bit, no parity, and should be set to the highest possible baud rate, generally the highest baud rate will be used automatically. Regardless of the baud rate setting, the communication is always at maximum speed.

- **COM(1) or /dev/ttyACM0** : used for the TULIP Command Line Interface (CLI). Use a terminal program (for Windows I use Tabby or TeraTerm, but most of these tools will work just fine). This is the main interface to manage ROM images, plug/unplug modules and manage all aspects of the TULIP4041.
- **COM(2) or /dev/ttyACM1**: used for HP41 mcode/bus tracer output. Another instance of a terminal program should be connected to this COM(2) port to present the results. The TULIP4041 constantly monitors the HP41 system bus (where the modules are physically plugged) and provides an almost realtime stream of information with details about bus activity. CLI commands are used to control if and how the data is presented and filtered. Default is no tracing active at all. Tracing can have impact on the performance of the TULIP4041. Tracing uses memory based buffers with limited capacity and filtering should be used to prevent overflows of these buffers. Overflows are visualized in the trace results and do not impact the operation of the TULIP4041.
- **COM(3) or /dev/ttyACM2** : used for HP-IL communication. The TULIP4041 can emulate the HP-IL ROM (with printer) and virtual HP-IL devices on a host computer (ILPer or PyILPer) can be connected to COM(3) to offer a large number of virtual devices. Internally the TULIP4041 emulates a PILBox, the host computer will see a PILBox connected. When a host computer is not connected the firmware will automatically simulate a closed HP-IL loop. When a host is connected but this is not a valid ILPer or pyILPer virtual HP-IL device the loop will appear open.
- **COM(4) or /dev/ttyACM3**: used for HP-IL frame tracing, much like an HP-IL scope. A terminal program should be connected to this port to present the results. In addition to HP-IL frames the serial communication to and from the emulated PILBox are shown (when enabled).
- **COM(5) or /dev/ttyACM4** : used for printer output from the HP82143A printer ROM (PRINTER 1E). When this ROM is plugged the output is sent to this port and the HP82240 simulator (for Windows) can be used for

presenting the results. The HP82240 simulator should be set to HP82143A mode for correct results, including graphics. With the CLI the printer characteristics can be changed, for example TRACE/MAN/NORM mode.

Another serial port is available on GPIO 0 and 1, UART Tx and UART Rx. This serial port runs at 115200 baud, 8 databits, no parity and 1 stopbit. This port can be used to connect to a Pico Probe (or a USB to UART interface, connect only 3.3V equipment to the GPIO's!) and offers a very low level CLI that was used for development. In some cases low-level error messages are shown here. Use only when really needed in case of support. Do NOT use the offered FLASH programming functions! The function of this port may change in future firmware versions.

If a connection to one of the virtual serial ports is made or broken a message in the CLI will be shown.

9.2. Reset of the TULIP4041

A reset of the TULIP4041 can be achieved by removing all power and applying power again. Alternatively, connect the RUN pin on the Pico2 board to GND for a reset. The RUN pin is exposed on the TULIP-Module, and shorting this pad to GND will reset the unit. The CLI can be used to reset the TULIP4041 using the ***system REBOOT*** command.

9.3. Signalling LED

The TULIP has a blue signalling LED that is driven by GPIO25, just as on the standard Pico board. In BOOTSEL mode this LED shows activity whenever a firmware file is copied, in normal mode when the TULIP firmware is running it flashes 2 times upon startup. After that the LED reflects the state of PWO of the HP41 calculator and lights up whenever the calculator is running. This LED functions as a drive activity indicator when downloading firmware in BOOTSEL mode.

9.4. Micro SD card

The micro SD card is provided by the user and can have any reasonable size larger than 2 GByte (the used FAT library does not support SDSC cards, only SDHC and better). It should be formatted with the FAT32 or exFAT (recommended) file system and contains the following information:

- Root directory with the LIF container files (for use with a virtual HP-IL drive) and configuration backup files
- (optional) MOD directory containing your own repository of .MOD files (MOD1 and MOD2)
- (optional) ROM directory containing your own repository of .ROM files
- RAW directory containing your collection of .RAW files (HP41 user programs). These files can be accessed only by the special TULIP ROM (not available yet).
- In practice use your own directory with a repository of ROM and MOD files that you want to have available in FLASH for plugging. Firmware functions allow easy mass import for the uSD card into FLASH and upgrade in case of changes
- ROM images must be imported in FLASH (using the CLI) before these images can be virtually plugged

The file system now supports the option of changing the active directory with the ***cd*** command. Further commands like ***dir***, ***import*** and ***export*** work on the current directory.

IMPORTANT: some high speed or high capacity micros SD cards do not work on the TULIP as these cannot be initialized using the simple SPI interface.

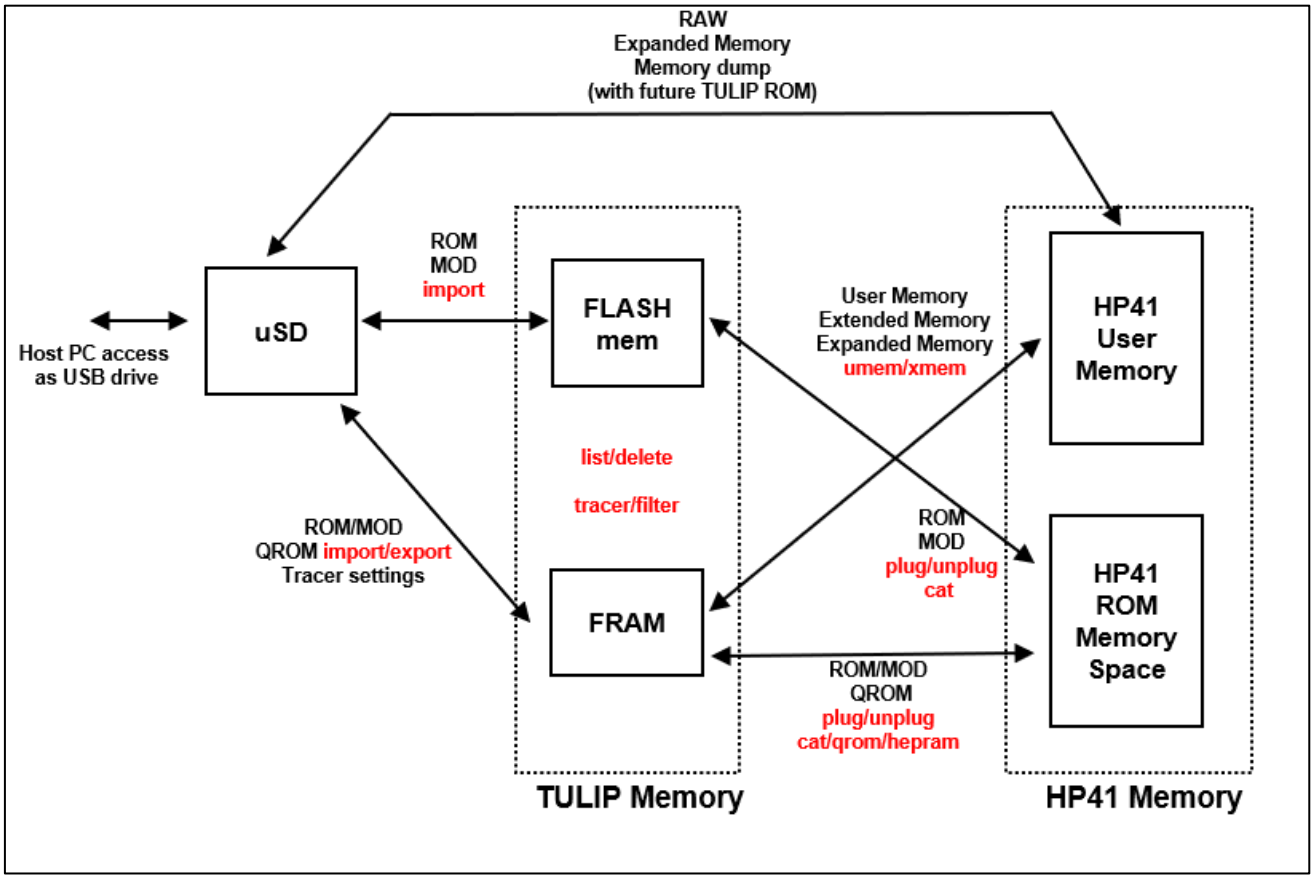
Please understand the following:

- **IMPORTANT**: The name of a directory or file in the micro SD card should not contain any spaces. This is not supported by the functions in the firmware unless the name is enclosed in quotes, which of course is inconvenient
- The microSD card must have a size of at least 2 GByte (SDHC and SDXC type cards), the FatFS library used in the firmware does not support SDSC type cards
- Best plug or unplug a micro SD card only when the TULIP4041 is not powered
- Format the card on a host computer according to the FAT(32) or exFAT file system. The TULIP4041 firmware uses the FatFS library to manage the files on the card, and only FAT or exFAT file systems are supported
- The exFAT file system is recommended, this is much faster that the FAT file system, especially when connecting the TULIP to a host computer
- Filenames of files that are to be imported in FLASH or FRAM are limited to 31 characters (and that includes the extension and dot) and should not have any spaces
- (optional) After formatting create a MOD and ROM subdirectory in the root directory. A RAW subdirectory or a LIF container is optional.
- (optional) Any MOD images that you plan to use should be put in the MOD subdirectory
- (optional) Any ROM images that you plan to use should be put in the ROM subdirectory
- Best practice is to copy all your favorite ROM and MOD files that you wish to be pluggable in a single directory
- Put the micro SD card in the slot on the TULIP device and power the device
- Use the ***sdcard mount*** in the CLI to activate the card, although this normally happens automatically
- Use the ***dir*** command, followed by the directory to list the contents of the card
- The micro SD card is connected to the processor with a single channel SPI connection which is not very fast
- The micro SD card is exposed as a removable USB drive on your host computer. Use the command ***sdcard connect*** to make the card visible (should be automatic after startup), and ***sdcard eject*** to remove the card from the host PC. After ejecting the drive itself is still visible on the host computer but cannot be used

10. ROM images, User memory and the File System

The handling of ROM images is an essential part of the TULIP4041 system to allow these images to be visible to the HP41. The main source of any ROM image is the micro SD card, this is the place where you collect your ROM and MOD files. The micro SD card is easily accessible from a host computer as a disk drive and you can copy your ROM repository to a directory on the card.

The TULIP firmware cannot delete, copy or rename files in the micro SD card, this must be done by the host computer



In order to be (virtually) pluggable into the HP41 memory space a ROM image must be present in FLASH or FRAM memory. The ***import*** command (just like on the DM41X) copies a file from the uSD card to FLASH or FRAM. ROM images containing writeable QROM should be imported into FRAM memory.

The image type can be a MOD or ROM file. Both FRAM and FLASH can contain other file types:

.MOD	MOD1/MOD2 files (MOD2 files are currently not supported)
.ROM	ROM files (only uncompressed ROM files are supported)
.XPM	Expanded memory (only in FRAM) (in development)

The system files in FRAM are created and initialized when the TULIP first starts (this will typically be your shipping configuration) and are at fixed locations in FRAM.

TULIP_Config.TCF	TULIP Configuration File
TULIP_Module.TMM	TULIP ROM mapping
TULIP_Tracer.TRF	TULIP Trace Filters
User_MModule.UMM	User Memory Modules (initialized to zero)
Extd_MModule.XMM	Extended Memory Modules (initialized to zero)

10.1. Importing files and the File Manager

The files in FLASH and FRAM are handled by a very basic FLASH/FRAM File Manager (FFM). It handles the file type, file name (up to 31 characters including the extension and the dot), file size and position of the next file. The files can be programmed from the micro SD card in FLASH or FRAM with the ***import*** command in the CLI. If there is no more room or a file with the same name already exists an error message will be given (unless the update option is used). Batch importing can be used to import all supported filetypes from a subdirectory. Before it can be used, the FFM must initialize storage with the ***flash INIT*** command. This creates the start of the file chain in FLASH by creating a file name "TULIP4041 FLASH HEADER". When this file is present the file system in FLASH is initialized.

The same is true for the file system in FRAM, and it contains a number of extra files which are created during initialization with the ***fram INIT*** command.. These are the following:

TULIP_Config.TCF	TULIP Configuration File
TULIP_Module.TMM	TULIP ROM mapping
TULIP_Tracer.TRF	TULIP Trace Filters
User_MModule.UMM	User Memory Modules (initialized to zero)
Extd_MModule.XMM	Extended Memory Modules (initialized to zero)

The above files must always be present in the given order and at fixed addresses to guarantee fast access to the data.

A file can be removed from FLASH or FRAM with the ***delete*** command, except for the system files created by the firmware. Be aware that the image must be unplugged before deleting.

To save a ROM or MOD image back to the micro SD card, use the ***export*** command. This will typically be useful for QROM images that you have modified (with HEPAX for example). You can then easily copy the exported file to your host computer for further actions. The other files types can be imported and exported for archiving or sharing. The ***export*** command only works with files in QROM/FRAM space.

To find out which files are in FLASH and/or FRAM use the ***list*** command. To see which modules are already plugged use the ***cat*** command (just like your HP41). The ***dir*** command is used to list the files on the micro SD card.

Importing, exporting and deleting files can be done only when the calculator is off or in light sleep (PWO is low). Be aware that sometimes the HP41 wakes up by itself (due to an alarm or other peripheral) which may corrupt the file. Filenames must be unique in FLASH and FRAM, you cannot have the same filename in both FLASH and FRAM!

For efficient storage in FLASH memory the granularity of files storage is 256 bytes. FLASH memory has a limited number of write cycles and wear may occur on memory locations that are frequently written to. According to the device datasheet the number or erase/write cycles should be at least 100.000. Theoretically this will give you

about 273 years if you erase a FLASH sector every day. The file manager has some limited handling to prevent wear in FLASH memory. If the memory starts to develop problems the FLASH memory chip must be replaced. FLASH and FRAM storage may become fragmented and currently no tools for defragmentation are planned. Best approach when defragmentation becomes an issue is to save (export) all relevant files to the micro SD card and re-initialized storage.

IMPORTANT: During any FLASH erase or programming operation the TULIP must remain powered. Power loss may result in loss of data and/or a corrupt file system. During programming some FLASH contents are temporarily saved in RAM.

The size of a file in the flash system can be estimated as follows:

- Take the size of the file in bytes
- Add 40 bytes for the file system header
- Round up to the next 256-byte multiple

To prevent wear on the FLASH memory most new files are placed at the end of the existing files. Gaps (which can exists after deleting files) can be filled by new imported files. Files can also be updated without prior deleting, in this case the size and type must be identical. Deleting a file means that the file is not completely erased, but the file type is set to erased and the file chain is kept intact. Only when a new file is imported in the space of a deleted file will the chain be updated. The ***import*** function will search in the FLASH File System for the smallest space where the new file will fit, otherwise the file will be programmed at the end of the chain. A file is imported by default in FLASH, importing in FRAM must be done by using the ***qrom*** option.

Deleting files and subsequent programming of new files can lead to fragmentation of the Flash File System, and there are no tools to resolve this fragmentation. Given the size of FLASH (especially in the Module version) this will not become a common issue. Fragmentation can be resolved only by completely erasing all FLASH and importing the needed files again.

The FLASH memory can be completely erased (nuked) with the ***flash NUKEALL*** command. This erases the complete area reserved for the file system. It will not erase the part of FLASH where the firmware is. After erasing the filesystem must be initialized with the command ***flash INIT,*** this will initialize the filesystem by creating the header file and the necessary system files. When the file chain is corrupted or other issues are resent this is the only way to resolve this.

Before use the FRAM is automatically initialized. Both FLASH and FRAM can be inspected with the ***flash dump*** and ***fram dump*** commands. Like FLASH memory, FRAM can be fully erased with the ***fram NUKEALL*** command, and initialized with the ***fram INIT*** command.

FRAM supports an almost infinite number of write cycles. Since FRAM is used constantly by a running HP41 operations on FRAM can only be done when the calculator is not running.

Due to its limited capacity (256 Kbytes) only files that really need the random write access should be placed in FRAM. This includes a number of system files (requiring about 30 Kbyte) and files containing QROM images (see next paragraph). Any file can be forced to be imported in FRAM, but only relevant files (QROM and some system files) should be imported in FRAM. By default a file is always imported in FLASH. A .ROM file can be manually forced in FLASH or FRAM if needed. Fragmentation can occur in FRAM as well.

Sometimes a ROM image is updated by the developer. To facilitate the update process it is possible to replace an existing file in FLASH or FRAM with an updated version of the image under the following conditions:

- The file name is identical
- The file size is identical
- The contents are different
- There is no need to first unplug the ROM image if it was plugged if:
  - The exact same bank assignment is used in case of a multi-bank module
  - The exact same page order is used in case of a multi-page image
  - There are no changes in the hardware support of a MOD file if it is plugged

When a ROM image is updated in the way described above multiple FLASH sectors may be erased containing parts of the ROM image to be updated. When multiple ROM images are updated a single FLASH sector may be erased multiple times thus increasing the risk or wear. This does not apply when updating ROM images in FRAM.

A special variation of the import function can be used for batch programming your complete ROM and MOD repository stored in a subdirectory on the micro SD card to FLASH. This saves you a lot of manual work. Prepare the directory with the files you intend to use and make an estimate of the amount of space used in the FLASH file system, to check if it will fit. Best is to start with fully erased (and initialized) FLASH, but that is not really necessary. In case of any updated files, you can simply run this command with the UPDATE option. The command to be used is ***import [directory] ALL***. This will program all MOD and ROM files in the subdirectory "directory" to FLASH with the following rules:

- Existing files will be skipped
- When the ***UPDATE*** parameter is used the existing files will be replaced under the same rules as the single file update
- New files will be appended to the file system (or inserted in free slots)
- The import will stop after all files have been imported or when the end of the FLASH file system is reached
- Files will only be imported into FLASH, never in FRAM

When importing a single file the filename in the Flash File System will be in the case as you typed on the command line. When importing multiple files, the case will be as it appears in the directory listing on the micro SD card. The mass update option is not implemented for FRAM.

After importing one or more files the operation can be verified with the ***compare*** option. This can be used on a single file or on a complete directory. A use case is to check which files may need to be updated. As an extra the compare option will indicate for every file if it can be simply reprogrammed, or if a flash erase must be done first. When using the ***compare*** option the contents of FLASH or FRAM will never be modified.

Details of the commands to manipulate ROM images are described in the CLI reference. In short here is an overview of the relevant CLI commands:

delete [filename]

list <filename>

dir <directory>

import [directory] [UPDATE] [ALL]

import [filename

import filename qrom

10.2. Exporting files

After changing the contents of a QROM file, for example when developing mcode or after storing data in a HEPRAM Page, the file can be exported from FRAM (exporting from FLASH is not supported) to the uSD card for offline storage, documentation or exchange with another calculator or emulator.

export [file]	saves the named file (ROM or MOD) to the current directory on the uSD card
export [file] dir/filenm	Saved the file to a new file, optionally in another target directory

10.3. Files containing QROM

QROM stands for Quasi-ROM. This is a ROM image that is actually in RAM and can be written to with a special HP41 instruction (WROM, 0x040). This is the way the MLDL's from the good old days work, and this also works in the HEPAX module, NoVRAM, MLDL2000 and the HP41CL. To be able to do the write instruction the ROM image must be placed in FRAM. The WROM instruction is a NOP to the HP41 processor and implemented in the TULIP firmware. The ROM image must be plugged before it can be accessed by the HP41. A QROM image is recognized by the RAM indicator in the page metadata in case of a MOD file. For a file to be used as QROM it must be imported in FRAM by using ***qrom*** option with the ***import*** command, the FRAM behaves like a kind of RAM.

Any ROM image in FRAM can be written to by the WROM (hex 040) instruction if it is plugged to the HP41 (if the Page is not write-protected). Using this instruction on an image in FLASH will have no effect. As with most MLDL type devices a QROM image can be write protected and also read protected. Protecting from reading by the HP41 can be useful to prevent the HP41 from responding in an undesired way to the QROM contents. This used to be the case when RAM chips were used in MLDL type devices. When not initialized the uninitialized RAM chips contains random data which are a guarantee for erratic calculator behavior. By disabling reads from the calculator, but by enabling writes the RAM contents can be initialized to zero (using a function in a specialized MLDL support ROM) to make reading safe. Normally this should not be a problem when one of the empty ROM or MOD files is plugged.

As a special features it is possible to have a ROM enabled for reading, but have the system polling points disabled for a Page. This can be very useful when developing mcode which uses these polling points.

The ***qrom*** command is used for setting specific parameters of plugged modules.

qrom <status>	list the status of the plugged modules, same as *cat*
qrom qrom p	toggle the QROM status of a plugged Page and all its banks
qrom enable p	toggle read enable, all banks in the Page
qrom wprot p	toggle write protect (only for FRAM), all banks in the Page
qrom hepram p	toggle the HEPAX marking of the Page (see the paragraph about HEPAX emulation)
qrom clear p <b>	clear the complete Page to zero, indicate bank in needed
qrom polling p	toggle the read enable of the system polling entries 0xFF4..0xFFA in Page p, this applies also to FLASH based Pages!

In all cases x is the Page number in hex, n is the bank number 1..4, default is bank 1

Keep in mind that the read and write enable status is a function of a plugged Page, not of a ROM image. When you unplug the status is lost. When a ROM image is plugged from FRAM, it is enabled by default for only reading and it will be write protected. Furthermore the file is not marked as QROM. These settings must be changed with the ***qrom*** command.

If this is a problem, simply plug the module while the calculator is off (good practice anyway) and adjust the settings before switching the calculator on.

A .ROM file does not have metadata and the user must ensure correct settings. It is very well possible to virtually plug a file multiple times in the HP41,. This can be useful in some advanced cases. Keep in mind that this refers to the same file, meaning that the change in one HP41 Page also results in a change in the other Page where the file is plugged. If you intend to use multiple QROM pages you must ensure that different files are plugged!

To optimize storage in FRAM it is not recommended to use MOD files with a mix of QROM and regular ROM images (for example some existing HEPAX MOD files). This simply takes up space in FRAM that could be used for other purposes. On the other hand the FRAM offers space for about 35 MOD or 28 ROM files.

10.4. Plugging ROM images

In order to make a ROM image visible to the HP41 it must first be (virtually) plugged in one of the ports. When a ROM image belongs to a peripheral (Printer or HP-IL module for example) or contains special hardware (like the HEPAX module) the hardware support for the special function must be enabled. For a ROM image in a .MOD file this is fully automatic in case the hardware is supported, but for a .ROM file this must be done by the user. The advantage of a .MOD file is also that multi-page images (like HEPAX or the Advantage ROM) are handled automatic. The automatic handling can only work if the TULIP knows if any physical modules are already plugged, and that cannot be done automatically. Therefore any modules that are physically plugged or already present in the HP41 (for example in the HP41CX) must be made known to the TULIP with the ***reserve*** command. This can only be done if you know the page it is plugged in. Pages 0, 1, 2 and 3 are always reserved for the HP41 system ROMS. Use the ***list [rom name]*** command to get details of the ROM or MOD file you intend to plug.

reserve [P] <name>	where P is the physical page number in hex. Optionally type the name (or any other descriptive text) of the module, this is used in the *list* command. This name or text cannot contain any spaces, unless it is enclosed by quotes (“)
reserve cx	if you have an HP41CX to reserve Page 3 and 5.
reserve printer	to reserve Page 6 (for the HP82143A, IR or HP-IL Printer module)
reserve hpil	to reserve Page 7 if you have the real HP-IL module plugged
reserve clear [P]	clear the reservation in the named port, port number in hex
reserve clear all	clear all reservations. The reservations for the system Pages cannot be cleared

Port	Page	Address	All models	HP41CX	Typical HP41 use
4	F	0xF000 - 0xFFFF	Port 4, upper Page	Port 4, upper Page	
	E	0xE000 - 0xEFFF	Port 4, lower Page	Port 4, lower Page	cardreader
3	D	0xD000 - 0xDFFF	Port 3, upper Page	Port 3, upper Page	
	C	0xC000 - 0xCFFF	Port 3, lower Page	Port 3, lower Page	wand (if cardreader plugged)
2	B	0xB000 - 0xBFFF	Port 2, upper Page	Port 2, upper Page	
	A	0xA000 - 0xAFFF	Port 2, lower Page	Port 2, lower Page	
1	9	0x9000 - 0x9FFF	Port 1, upper Page	Port 1, upper Page	
	8	0x8000 - 0x8FFF	Port 1, lower Page	Port 1, lower Page	
	7	0x7000 - 0x7FFF	HP-IL Module	HP-IL Module	
	6	0x6000 - 0x6FFF	(IR)PRINTER/IL Printer	(IR)PRINTER/IL Printer	
	5	0x5000 - 0x5FFF	TIME Module	CX TIME Module CX FNS bank 2	
	4	0x4000 - 0x4FFF	TAKE OVER ROM disabled IL Printer	TAKE OVER ROM disabled IL Printer	Service ROM, 4LIB
	3	0x3000 - 0x3FFF		CX FNS – bank 1	
	2	0x2000 - 0x2FFF	HP41C OS – ROM2	HP41CX OS – ROM2	
	1	0x1000 - 0x1FFF	HP41C OS – ROM1	HP41CX OS – ROM1	
	0	0x0000 - 0x0FFF	HP41C OS – ROM0	HP41CX OS – ROM0	

Regular User ROM space
Fixed module page if plugged
Be careful here
HP41CL/CX fixed pages



In case of an HP41CX, you must also inform the TULIP about the standard ROMs. When you use an HP41CL, remember that this is always an HP41CX configuration, and that any virtual ROM images in the HP41CL must also be reserved. This reservation of a physical module only prevents the TULIP from plugging ROM images in the reserved Page.

To plug a ROM image in the TULIP simply type the ***plug*** command with the name of the file with the extension. For a .MOD file nothing else is needed. If you have correctly informed the TULIP about any physical modules the ROM image will be automatically plugged in the best possible Page according to the MOD handling algorithm. For a .ROM file you must indicate a Page and optionally a Bank number. But if you do not enter a Page number the firmware will plug the ROM image anyway in the first available Page starting at Page 8. An error message will be shown if the ROM image cannot be plugged, for example when the Page is already taken.

Care should be taken to plug just any ROM image in Page 6 as it is designed for the Printer and some entries are called regularly by the system ROMs. HEPAX and the OSX3 ROM images are known to work here.

Plugging a .ROM file means that you must provide the target Page. The Bank must be entered when the image must be in any of the banks 2..4. When plugging multi-bank ROM images bank 1 should be plugged last unless you are certain that the calculator will not run between the ***plug*** commands.

Summary of commands to plug a module:

plug [filename] [p]	plug the ROM image in Page p (p in hex)
plug [filename]	no Page number required, will plug in the first available Page. Use in combination with the ***reserve*** command

plug [filename] T use this for testing in which Page a file will be plugged, plugging will not take place (.ROM files only, not for .MOD files)

plug [filename] p b force the ROM image in Page p and bank b (b=1..4), only for .ROM files. This is the only plug variation that allows overplugging, meaning that it is not necessary to unplug the ROM image in the Page/Bank first.

The plugged ROM images are maintained in a rom map by the TULIP in the CModule class and this is stored in FRAM.

When plugging MOD files some limitations apply:

- The ROM images in a MOD file will be plugged in the order as they appear in the MOD file. If an image causes trouble (fixed page already taken or unsupported hardware) the image will be refused, but the previous images from the MOD remain plugged, these must be manually unplugged if so desired
- Plugging HEPAX is supported, but the HEPAX instruction for moving a page (ROMBLK) is not supported. When using HEPAX use the modified HEPAX-4H image (same as for the HP41CL)

Currently only MOD files with the following hardware are supported. When these are plugged the respective hardware emulation is enabled.

- HP-IL
- Printer HP82143A
- HEPAX (only the HEPAX WPTOG instruction is supported, ROM BLK is not)
- Wand (Barcode reader)

The CModule class maintains the following information:

- Filename and address in the FLASH/FRAM file system. For a multi-bank module this is the filename of Bank 1 (in case of a .ROM image)
- Module type: MOD1, MOD2, ROM, or physical (to manage real modules plugged)
- Pointer to each Bank
- Current active Bank and a Sticky bit for ZEPROM emulation
- Hardware type (HEPAX, QROM)
- Module status: read/write enabled, QROM, DIRTY (QROM changes to be done)

To view the ROM images that are plugged use ***cat*** command:

cat show the contents of the rom map [filename], or the active rom map

cat [X] show details of the ROM plugged in Page X (in hex) with a hex dump of that ROM

10.5. Bank switching in the TULIP

Bank Switching is a delicate topic, as it is traditionally handled by the hardware in a module, and different implementations exist. It is therefore a bit tricky to generalize the behavior of bank switching, but necessary to emulate this as accurate as possible in the TULIP firmware. The TULIP monitors the HP41 bus for the bank switching instructions ENBANKx (x=1..4) for all banks for including this information in the tracer and for selecting the correct word from a ROM image. The active bank is maintained for every single page and is reset to Bank 1 when the calculator goes in STANDBY mode.

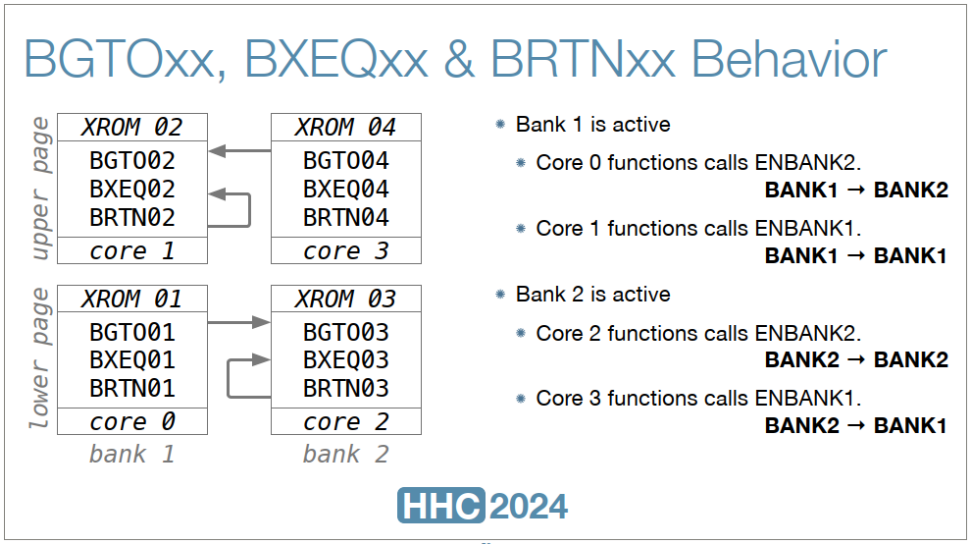
ENBANKx instruction in Page 3 switches banks in Page 5 (HP41CX behavior), used for the tracer

ENBANKx instruction in the Port Pages (Pages 8..F) switches banks in all Pages in that Port. For example switching a Bank in Page 9 also switches banks in Page 8.

ENBANKx instruction in any other Page (4..7) switches banks in that Page only. This enables bank switching in Page 4 and in Page 5 (if a CX is not used)

When a Bank is switched in a PORT and the other Page does not have an active ROM image plugged in that Bank then always the information from Bank 1 is fetched. This may conflict with some specific hardware modules.

A special case of bank switching is implemented for emulation of the ZEPROM. This must be specifically enabled for a Port with the ***emulate zeprom P*** command, and the ZEPROM ROM images must be manually plugged in the correct Page and Bank, the ROM images are not automatically placed like in the real ZEPROM hardware. The main feature of the ZEPROM bank switching is that the banks are sticky and are not reset to Bank 1 when entering STANDBY mode or DEEP SLEEP. To achieve this the bank switching mechanism in the TULIP firmware in the active Page has the 'sticky' bit set, and upon a bank change marks the Page as dirty. On the next PWO event (calculator going in Light or Deep Sleep) the current state is then saved to FRAM if the dirty bit was set.



EXPLANATION OF ZEPROM BANK SWITCHING (BY SYLVAIN COTE)

The TULIP emulation of the ZEPROM follows the explanation in the figure above (thanks Sylvain). More details in his presentation at HHC2024. Be aware that the Sticky Bank switching can be enabled even if the ZEPROM ROM is not plugged.

10.6. Embedded ROM images

The TULIP has a number of built-in ROMs or Embedded ROMs. These are fully included within the firmware file and do not have to be imported, and also are not visible with the ***list*** command. These Embedded ROMs are:

- HP-IL and HP-IL Printer ROM
- HP82143A Printer ROM

When plugging these ROM images the Page and hardware emulation is automatically selected (and disabled when unplugging). The ROM images are identical to the original devices.

plug hpil plugs the Embedded HP-IL ROM image in Page 7 and enabled HP-Il emulation

plug ilprinter	plugs the Embedded HP-IL Printer image in Page 6 and enables HP-Il emulation. The HP-IL ROM must be plugged first!
plug printer	plugs the HP82143A printer ROM in Page 6 and enables the emulation for this

Unplugging is done with the ***unplug*** command, and the correct Page number must be given. Emulation will then be disabled for that peripheral.

10.7. Hardware emulation

In order to support emulation of specific hardware this has to be enabled. In case you are using a real HP-IL module the HP-IL module in the TULIP **must** be switched off to prevent conflicts. When using the Embedded ROM images the emulation is automatic. It is very well possible to import the HP-IL ROM image and plug it in Page 7 (**this ROM must be plugged in Page 7**!). In that case the emulation inside the TULIP of the HP-IL hardware registers must be manually enabled with the ***emulate*** command.

emulate	show the status of emulation
emulate hpil	toggle HPIL hardware emulation
emulate printer	toggle HP82143A printer emulation
emulate wand	toggle HP82153A Wand (barcode reader) emulation
emulate zeprom P	toggle ZEPROM emulation in Page P (hex) for sticky bank switching See the separate topic on bank switching

Do NOT manually switch off emulation while a ROM is still plugged that needs the emulation. This will lead to unexpected events.

IMPORTANT: when unplugging a ROM image with specific hardware emulation, this emulation is switched off only for the Printer and HP-IL module. Other hardware types must have their emulation manually disabled.

10.8. Unplugging ROM images

To remove a module and make it invisible to the HP41 you can use the ***unplug*** command. This simply removes a ROM image from the rom map. Unplugging is done by Page number. If you ***unplug*** from a Page which is reserved for a physical module this will be unplugged from the TULIP, remember to physically unplug as well. The contents of the ROM or MOD file will not be removed, and any changes done in a QROM Page will still be present in the file in FRAM and can be exported to the uSD card.

unplug p	unplug all Banks in the indicated Page p (p in hex)
unplug p b	unplug the Bank b in Page p
unplug all	unplug all plugged ROMs except reserved Pages
unplug ALL	unplug all ROMs including all reserved Pages (except Page 0..3)

HP41 User Memory

The HP41 User Memory is the calculators memory that is used for the normal registers, stack, status registers and program memory. This is a single address space of 56-bit registers and is totally independent from the memory space where the ROM images and the HP41 operating system resides. The HP41 user can access the memory with the STO and RCL instructions, by accessing the stack, using Extended Memory functions and by entering programs.

HP41 Address	Name	Used for
0xC00 – 0xCFF	Expanded Memory 3	Expanded Memory in HP41CL/MAXX/TULIP
0x800 – 0x8FF	Expanded Memory 2	Expanded Memory in HP41CL/MAXX/TULIP
0x400 – 0x4FF	Expanded Memory 1	Expanded Memory in HP41CL/MAXX/TULIP
0x301 – 0x3EF	X-Memory	X-memory Module 2
0x201 – 0x2EF	X-Memory	X-memory Module 1
0x100 – 0x1FF	User Memory	User Registers HP41CV + CX, Single or Quad Memory Module Programs, Key Assignments, Alarms, Buffers
0x0C0 – 0x0FF	User Memory	User Registers HP41C, always available Programs, Key Assignments, Alarms, Buffers
0x040 – 0x0BF	X-Memory	X-memory Module 0, in X-Functions module
[not used]		Non-existing memory
0x000 – 0x00F	HP41 Status registers	User stack, ALPHA, flags, return stack, SIZE info, OS use

A part of the User Memory is Extended Memory, managed by the Extended Functions ROM (built-in in the HP41CX). Direct access to this memory in mcode is done using the RAMSLCT (DADD=C) instruction to select a block of 16 registers, and with the READ and WRIT instructions. All access is always using all 56 bits, and the address is 10 bits wide, limiting the number of registers to 1024 (hex 400). The HP41 memory map contains a number of gaps (non-existent registers) which are vital for the correct operation of the HP41. Extended Memory is maintained in files in FRAM and this file is created when the TULIP starts for the first time and it reserves the full Extended Memory in a single file, even if you do not plan to use it.

With the introduction of the HP41CL Monte Dalrymple defined Expanded Memory. This is User Memory beyond the original User Memory by using a 12-bit address and a new instruction EADD=C (0x0C0) to select this memory. With this method there are now 4096 registers available. Please refer to the HP41CL and MAXX documentation for further details. *This type of memory is not yet supported in the TULIP firmware.*

Depending on the type of HP41 you may have several types of User Memory already available in the calculator, the table below indicates what you would need extra to get a full configuration. The Extended Functions module (ROM or MOD file) must be plugged separately if that is not already in your calculator.

xmem <status>	shows the current status
xmem X Y	X=0,1, Y=0..2 Plug the Extended Memory module(s) where X is the memory in the Extended Functions module, and Y is the number of Extended Memory modules
xmem 0 2	plug Extended Memory modules 1 and 2 and no Extended Functions memory (in case you have a physical Extended Functions module plugged or an HP41CX)

User Memory modules are handled in the same way as Extended Memory. The file contain these registers is automatically created in FRAM and can be managed with the following CLI commands:

umem <status> shows the current status

umem N N=0..4, plugs 0 to 4 User Memory Modules, using 0 unplugs all memory modules

umem 4 is the equivalent of plugging a Quad Memory module

It is not possible to plug User Memory in such a way that a gap in memory is created.

HP41 type	**Built-in**	**Possible with TULIP**
HP41C	0x000 – 0x0FF Only basic memory	1..4 single Memory modules Quad Memory Module Extended Functions + 2* Extended Memory Expanded Memory Extended Functions ROM or MOD file
HP41CV	0x000 – 0x0FF 0x100 – 0x1FF Basic memory+ Quad Memory	Extended Functions + 2* Extended Memory Expanded Memory Extended Functions ROM or MOD file
HP41CX	0x000 – 0xCFF 0x100 – 0x1FF 0x040 – 0x0BF Basic memory + Quad Memory Extended Functions + Basic Extended Memory	2* Extended Memory Expanded Memory
HP41CL	Everything including Expanded Memory	Lucky you, you have everything already
HP41 (any model) with MAXX	Everything including Expanded Memory	Don't bother, it's all in the MAXX

Expanded Memory can only be used with special functions. These are available for the MAXX module and the HP41CL, but not yet for the TULIP. Use of it is optional.

User Memory is stored in a file in FRAM, this files is created when the FRAM file system is initialized. This file always has the maximum size for supporting a Quad module.

File types in FRAM:

.UMM User Memory
.EXT Extended memory
.EXP Expanded memory (not yet supported)

Summarizing for your situation:

HP41CL or HP41 (any model) with MAXX: there is really nothing to do, the CL or calculator with MAXX is fully loaded with all available Extended and Expanded memory

HP41CX: Extended Functions and basic Extended Memory is built in. Remove any existing Extended Memory modules and use ***xmem 0 2*** to get the maximum possible amount of Extended Memory

HP41C/CV with a physical Extended Functions: treat like an HP41CX

HP41C/CV: use ***plug EXT-FUNS.MOD***. Use ***umem status*** to find out how much Extended Memory you now have and use ***xmem 1 2*** to get the maximum amount of Extended Memory

HP41C: remove all physical memory modules (single or quad) and use ***umem 4*** to get the maximum amount of memory

HP41 type	CLI command	Result
HP41C	***umem 4***	Plug a Quad Memory module
HP41C + X-Functions module	***umem 4*** ***reserve 8 XFunctions*** ***xmem 0 2***	Plug a Quad Memory module Reserve the Page, assuming the Xfunctions module is plugged in Port 1 (Page 8) Add 2 Extended Memory Modules
HP41CV No modules	***plug ext-funs.mod*** ***xmem 1 2***	Plug Extended Functions virtual module (without memory) Plug X-Functions memory and 2 XMem modules
HP41CX	***xmem 0 2***	Plug Extended Memory 1+2
HP41CL	nothing to do	All memory already built in the HP41CL
HP41 with MAXX	nothing to do	All memory already built in the HP41 and in the MAXX

For fun or testing the amount of memory modules can be reduced or increased, but you may lose data or lock up your calculator.

After removing the TULIP containing User or Extended Memory from your HP41 do not power the calculator until the TULIP is plugged again. Otherwise a MEMORY LOST may occur.

10.9. Using the mcode tracer

For details about the workings of the mcode tracer, please refer to the section about the tracer in the TULIP Background document. The triggers are not (yet) implemented.

The mcode tracer is enabled whenever its serial port is connected. By default all samples are passed by the core1 critical loop to the queue, and the core0 software reads from the queue to display the samples. When the queue is full the core1 software will simply skip the sample, and the core0 software will miss a sample and indicate this with an O (for Overflow).

The tracer buffer has a default size of 5000 samples. This can be changed using the ***tracer buffer*** command. A reboot of the TULIP is necessary for the new buffer size to become effective. Maximum tracer buffer size is 10000 samples, this is limited by the amount of memory.

The ***tracer*** command is used to control the behavior of the tracer.

tracer or ***tracer status***	show the status of the tracer
tracer buffer <size>	change the size of the tracebuffer (reboot required)
tracer trace	toggle tracer enable/disable
tracer hpil	toggle HP-IL frame tracing to the ILSCOPE serial port
tracer pilbox	toggle tracing of PILBox serial traffic to the ILSCOPE port
tracer ilregs	toggle tracing of the HP-IL registers in the tracer
tracer filter	toggle filter enable/disable, when disabled all samples are passed

The tracer has a feature to filter samples, and uses an array of all possible addresses in the HP41 ROM Memory space to quickly decide what to do with a sample. Although this is memory intensive it reduces the load on the TULIP's microprocessor. The active filter is stored in FRAM in the file TULIP_Tracer.TRF and is copied to FRAM when the TULIP is powered and saved any time filter settings are changed.

The filters are controlled by the ***filter*** command. Two bits per address indicate one of the following options:

- Pass: Pass the sample at the given address. Default is to pass all samples.
- Block: samples at this address are blocked and not shown in the trace listing
- Trigger Start: samples that match a trigger address are marked in the listing, and traces are listed starting at this address. Traces are shown until the Trigger End address appears or until a given number of samples are shown. The contents of the pre-trigger buffer are shown before the Trigger Start sample
- Trigger End: Sampling stops after the Trigger End address or earlier if a number of samples is given (triggers are not yet implemented)

When using Pass or Block the order of the commands is important. For example to view only samples with a certain address range (within your own ROM for example) the first command must Block everything, and a later command can then pass a range:

filter block all	block all samples
filter pass p9	pass all samples in Page 9
or	
filter pass 9400 9600	to pass all samples between 0x9400 and 0x9600

A subsequent block command can be used to block a range within the range of passed addresses.

A number of standard addresses can be set automatically:

filter sysloop	toggle filtering or the following address ranges: `0x0098 - 0x00A1` RSTKB and RST05/RST10, reset and debounce `0x0177 - 0x0178` DRSY25, delay for debounce `0x089C - 0x089D` BLINK01, `0x0E9A - 0x0E9E` NLT10 wait for key to NULL `0x0EC9 - 0x0ECE` NULT10 NULL timer
filter	show the current active filters
filter block	show only the current active block filter
filter pass	show the current active pass filter
filter dump	dump the filter memory, used for debugging
filter save [filename.trf]	save the current active filters to a file on the uSD card
filter load [filename.trf]	load a file on the uSD card to become the active filter

Limitations:

- Pass, block and triggers apply to *all* banks of that Page
- Pass, block and triggers apply to both instruction fetch (SYNC high) and data fetch (SYNC LOW)

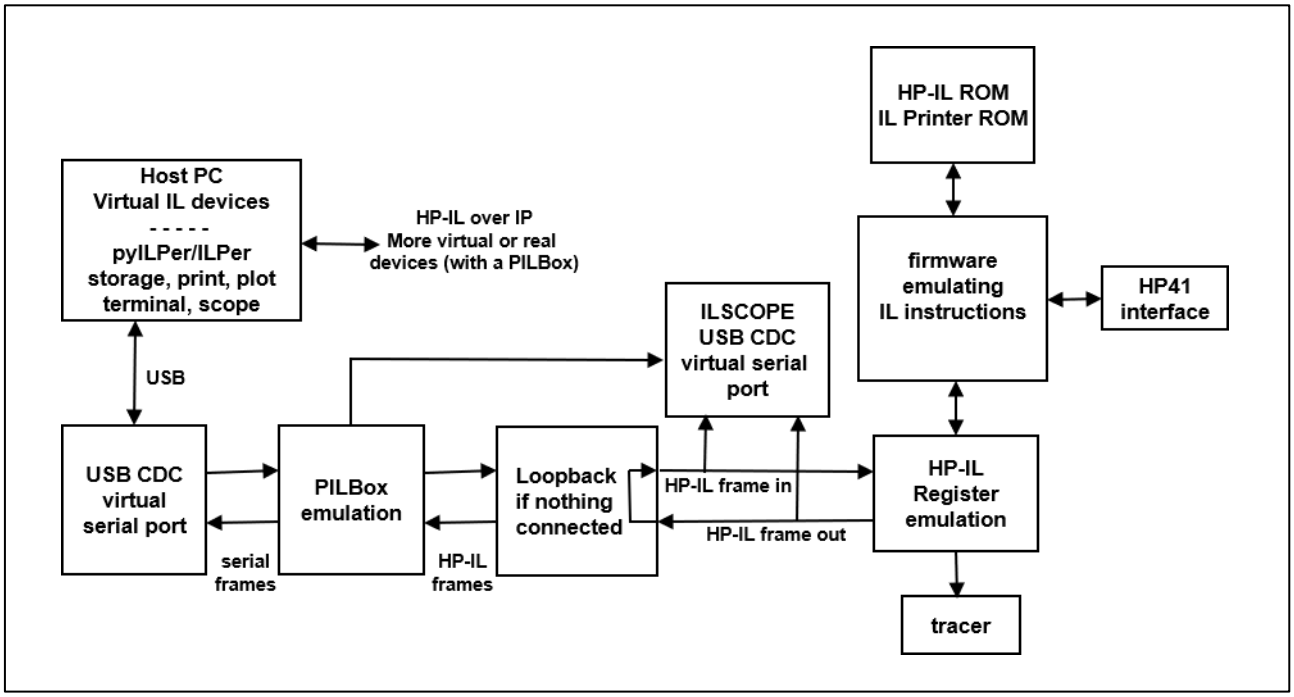
The tracer allows to set the size of the main trace buffer and the pre-trigger buffer. The default values are 5000 for the main tracer buffer, and 32 for the pre-trigger buffer. Keep in mind that the tracer is memory intensive, and one trace sample uses 35 bytes, this means that the default trace buffer of 5000 samples uses about 170 KByte. And total available memory is 520 Kbyte. When the size of the main trace buffer is changed a restart of the TULIP firmware is needed to ensure that the available RAM memory is not fragmented. The maximum size of the main trace buffer is 10000 samples. The size of the pre-trigger is fixed at 256 trace samples.

Pressing a key in the tracer window will enable or disable the tracer and pause the listing. Since the tracer will be disabled it will not continue at the halted address!

The trace window displays PWO events with the relative timing. Any change in the HP41 power state is shown before or after the trace. In case of many overflows the transition from Running to Light or Deep Sleep may be missed.

10.10. HP-IL emulation

The TULIP HP-IL emulates a virtual HP-IL loop. This means that it connects with a host computer via a USB virtual serial port to virtual HP-IL devices running on the host computer. There are provisions to expand the possibilities in future firmware versions. TULIP emulates the registers of the 82160A HP-IL module and the instructions to communicate with these registers and in addition uses the FI signals towards the HP41 system bus. The emulation is based on the V41 sources by Christoph Giesselink and EMU41 by Jean-Francois Garnier.



A (virtual) serial link is used for the connection with the host computer, and the translation of HP-IL frames to serial is that of the PIL Box (by Jean Francois Garnier), which is emulated by the TULIP4041. The host software for virtual HP-IL will see a PILBox connected. When no Host PC or virtual serial port is connected the HP-IL virtual loop is always closed internally. For more advanced functionality other ROMs can be plugged. The Plotter ROM is tested for example. For ease of use the HP-IL and IL Printer ROM images are embedded in the firmware (see the ***plug*** command)

In the current firmware version there are a few limitations:

- It is not yet possible to put the TULIP system in device mode (with the IL Development ROM for example)
- RFC/CMD frame handling as in the original PILBox is not implemented. As a result multiple RFC frames may be sent to the Virtual IL devices.
- AUTOIDY mode is not implemented

For debug and study purposes the HP-IL registers are sent every cycle to the tracer queue and can be shown in the tracer (using the tracer virtual serial port and a terminal emulator) The registers are shown only when there is a change. With the CLI ***tracer*** command this can be enabled or disabled. The tracer supports the HP-IL instructions in the disassembler.

HP-IL frames and PILBox serial traffic are sent to the ILSCOPE virtual serial port for analyzing or debugging your HP-IL applications. This can be controlled by the CLI ***tracer*** command.

When there is no connection with a serial device the HP-IL loop is closed. When there is a serial connection and this is not a virtual HP-IL device (like ILPer or pyILPer) then the loop is open, and any HP-IL operations will result in **TRANSMIT ERR**. Only when a virtual HP-IL device is connected the loop will be closed again. Changes in the status of the PILBox connection are shown in the CLI. In some cases a **TRANSMIT ERR** or a slow down of HP-IL traffic can occur when the TULIP firmware is heavily loaded with traffic on several virtual serial ports at the same time such as showing the tracer and the IL Scope at the same time.

The HP-IL printer can be disabled (like the little switch on the HP-IL module) by simply not plugging it or unplugging it. The IL Printer ROM will completely disappear and is *not* parked in Page 4, so Page 4 is available for other fun stuff. The HP82143 Printer ROM can be plugged instead and can co-exist with the HP-IL module.

10.11. HP82143A Printer Emulation

The HP82143A printer emulation is based on earlier experiences with the printer emulation for the HP41CL and the USB Printer module by Diego Diaz. Main source of knowledge is the document “HP82143A Printer Study” by Doug Wilder. Printer output is sent to one of the virtual serial ports for use by the HP82240 simulator (from Christoph Giesselink). This simulator must be put in the proper mode for the HP82143A printer. Graphics printing is supported. The printer output is also sent (if enabled with a jumper on the DevBoard) to the infrared LED.

The printer output can be selected with the ***printer output*** command. The bytes printed to the IR port are *not* compatible with the real HP82240 IR printer and are intended to be used with an IR receiver (serial or USB) connected to a host computer and the HP82240 simulator in HP82143A mode.

With the ***printer serial*** command the printer output can be cycled between HP82143A, ASCII serial or UTF-8 serial output. The two latter modes allow text-based printing to a serial terminal emulator without the need for a printer simulator, all graphics commands are ignored, the output closely resembles the real printer output with as good as possible character translation. In ASCII mode the character translation is limited but still useful for listing or user mode tracing. In HP82143A mode a simulator is needed on the host PC. ASCII and UTF-8 modes do not work with IR output. In case you have both serial and IR output enabled the outputs will not produce identical byte streams!

Please do not forget to put (virtual) paper in the printer (***printer paper***) and to power the printer (***printer power***). The ***printer mode*** command is used simulate the slider between normal, trace and manual mode. See the help functions to discover the workings. For ease of use the Printer ROM image is embedded in the firmware file, see the ***plug*** command. With the ***printer print*** and ***printer adv*** commands the press of the respective key is simulated.

When USB is not connected and the TULIP is powered only by the calculator the settings for printer power and printer paper are ignored, as this is not logical for infrared output. The IR output must be selected though in the CLI, this is not automatic.

Summary of the most important printer commands:

printer	(without argument) show the printer status and modes
printer output	cycle the output type: none, serial, IR or both
printer serial	cycle the serial mode between HP82143, pure ASCII or UTF-8, applies only to the serial output, not to IR
printer power	toggle the printer power status
printer mode	cycle between NORMAL, TRACE and MAN mode (like the slider on the printer)
printer paper	toggle Out Of Paper status
printer print	simulate a short press of the PRINT button
printer adv	simulate a short press of the ADV button

10.12. Using HEPAX in the TULIP4041

The emulation of the HEPAX module is somewhat limited and can be compared with how it works in the HP41CL. Most important difference with the original HEPAX hardware is that the HEPAX RAM Pages (HEPRAM) are not initialized by the HEPAX module itself.

It is recommended to use the HEPAX version 4H, a version modified from the original with some improvements. For HEPRAM, this can be installed using one of the various HEPRAM files. The HEPAX ROM keeps track of the HEPRAM pages by chaining these with information in reserved memory locations in the RAM pages. Normally this chain is initialized by the HAPX ROM itself, but that is not possible in the TULIP emulation. The initialization of the chain is now done with the ***hepram*** command. Without parameters this command gives an overview of the HEPRAM pages found (and the QROM pages not marked as HEPRAM) and shows if the pages are initialized or not. When not initialized then use the command ***hepram INIT*** to initialize the HEPRAM pages. Only HEPRAM pages that are specifically reserved for other use are not initialized, these files have a value other than 0x100 or 0x200 in the word at xFF3 (see references). With the command ***hepram reserve P*** a page (in hex) can be reserved, the word 0x3FF will be written to the location. To clear a reservation use ***hepram release P***. To include the page again in the HEPRAM chain simply run the ***hepram INIT*** command again after releasing the page.

The ***hepram INIT*** command preserves the contents of the pages, apart from updating the information needed for the HEPRAM chain. If the XROM number was zero this will also be written (any unused value starting from 0x0B). A possible issue is that the initialization command cannot detect XROM numbers of physical ROMs in the system. This must be resolved manually by writing an unused XROM number in the HEPRAM page causing the conflict.

The TULIP github page has a number of MOD files available with HEPAX and various HEPRAM sizes. These are not initialized. When exporting a MOD file with initialized HEPRAM pages this can be transferred to another HP41 only if the HEPRAM pages are in the exact same location, otherwise the initialization must be done again. In an HP41 without the TULIP this is best done with the HEPINI command in the HEPAX applications ROM (by Angel Martin, see references).

HEPRAM behaves different than ´regular´ QROM pages and HEPAX hardware is marked in the MOD file and in the TULIP internal ROM mapping. This allows the HEPAX special instruction WPTOG to be emulated, which is ignored by ´normal´ QROM pages. The HEPAX RAMBLK function can be executed form the HP41 keyboard and is effective in toggling the HEPRAM write protect. The ***hepram INIT*** command works only on HEPRAM pages. QROM pages can be marked as HEPRAM by using the ***qrom*** command and then initialized.

If desired the HEPRAM pages can be completed cleared to zero with the ***hepram CLRAM p*** command, where p is the Page number in hex. After that the initialization must be run again. It is possible to erase the last page in the HEPRAM chain while keeping the contents in the earlier pages. The HEPAX manual part 2 explains the details of this process.

Summarizing the HEPAX related commands:

hepram	(without argument) show the plugged HEPRAM pages
hepram status	show the plugged HEPRAM pages
hepram ramtog p	toggle the write protect of HEPRAM Page p
hepram reserve p	mark Page p to be excluded from the HEPRAM chain
hepram release p	mark Page P to be included in the HEPRAM chain
hepram INIT	(re) initialize the HEPRAM chain while preserving the contents as much as possible
hepram INITALL	initialize all HEPRAM including HEPRAM reserved Pages
hepram CLRAM p	erase HEPRAM Page p to all zeroes, ***hepram INIT*** must be run afterwards to initialize the HEPRAM chain

All page numbers are in hex

10.13. The HP82153A Barcode Reader (Wand)

TULIP enables emulation of the HP82153A Wand or barcode reader. The most important functions are reading barcode files for HP41 programs generated by the HP41UC program, the other use is simulation of the Wand Paper Keyboard.

For importing HP41 User programs (Focal) the CLI ***wand*** command is used. It can open a file on the uSD card with the .wnd format as generated by the HP41UC program (see references). This is a text based file with the barcodes decode as hexadecimal strings. This file is generated from a .raw file with the command:

hp41uc /r=infile<.raw> /w<=outfile<.wnd>> <"Program Title">

The output looks like the following:

```
Advent

Program Registers Needed: 160

Row 1 (1 - 5)
551001C000F4004144569C00A9158CCA
Row 2 (5 - 10)
DD114401F2004C22114190F3FB594F55
Row 3 (10 - 14)
3F1280524520494E2041209BF37E8966
```

Copy the output file to the TULIP uSD card (can be in a directory). After plugging the ROM image (best to use the latest version 1F) and enabling the emulation this file can be scanned. Make certain that the calculator is not running and has enough free registers for the program. The HP41 does not have to be switched ON. Like the real thing, the calculator will be automatically switched on when a barcode is scanned. If the input on the calculator fails the process on the TULIP can be interrupted by pressing X.

As an example use the following commands:

plug wand.mod	plugs the WAND ROM and enables emulation
wand scan [filename.wnd]	starts the scanning, can be interrupted by pressing X

The other use is emulation of the paper keyboard. This allows the user to enter commands in the TULIP CLI that are converted to virtual barcodes and executed as Wand keyboard functions. The user can enter individual commands, series of commands, ALPHA strings and numbers in the host computers keyboard with the ***w*** command. The HP41 functions can be entered in upper or lower case. The functions are forwarded to the HP41 as they are and there is no error checking done except that the function must exist in the list as given by the ***wand list*** command.

wand list	show a list of supported HP41 Paper Keyboard functions
w beep	executes the BEEP function, enters in a program when in PRGM mode
w STO 00	executes the function STO 00, 00 is entered as individual digits
w 156.98E3	enters a number in X or a program line. A number starts with a digit or a – sign and can have no spaces, other characters will abort the virtual scan
w -	executes the – (minus) function
w "ABC"	enters the string ABC in ALPHA. A double quote " indicates a string. All ascii characters can be used and will be converted to an 8-bit value. The string may contain spaces
w "ABC\40DE"	enters the string ABC@DE, the \ character enables entering a hex code. To enter a \ use \\. \+ is used to generate the APPEND character, and must be at the start of a string. Hex codes < 0x10 can be entered but will be converted to a space (seems to be a bug in the Wand ROM ??)
w PI STO 00 CLX	executes 3 functions or inserts these in a program when in program mode. Functions must be separated by spaces
w RCL IND 02	Executes the RCL IND 02 function
w 'xyz	Enters the string or character as single ALPHA character keys until the space. Only works with ASCII characters < 128 and escape characters can be used
w prgm lbl 'ABC	Enters PRGM mode and type the label LBL 'ABC, but does NOT press the ALPHA key to complete the label
w '	A single quote without anything else simulates the ALPHA key
w 'ABCD'	enters the ALPHA characters ABCD, the closing ' exits ALPHA mode

Some HP41 functions are aliased for convenience, for example ENTER^ can be also be used with ENTER. Not all possible commands are yet implemented. Be aware that some functions require a specific argument, for example ASN must have an ALPHA string as argument and requires a key to be pressed.

The ***w*** command send commands to the Wand emulation only when the calculator is not running. When the calculator is running the CLI will wait until the HP41 enters light or deep sleep mode. The wait can be interrupted by pressing any key. While waiting for the calculator to enter light or deep sleep the tracer and all communications functions (like HP-IL or printer) are interrupted and this may lead to HP-IL frame errors. As a result, it is not always possible to send functions to the HP41 like the real Wand. This is especially the case when running the WNDTST function of the HP41.

The following table shows the used escape characters for strings (note that the case matters). How the characters are displayed on your HP41 depends very much on the ROM version and type (C, CV, CX or CL). Many lower case characters will be displayed as starbursts. These work for the strings with single quote and double quote.

Escape code	**Result**	**Hex code**
\XX	Hex code XX	0xXX *
\\	\	0x5C
\S	sigma sign	0x7E
\+	append	0x7F
\t	hanging t	0x60
\#	not equal	0x1D
\s	space	0x20
\u	greek mu, or micro	0x0C **
\a	angle symbol	0x0D **
\’	single quote	0x27
\”	double quote	0x22 **

* Characters 0x00..0x0F are replaced by a space in strings with the double quote “

** not supported in ALPHA strings (starting with “)

For those wishing to experiment it is possible to send any code to the Wand emulation even when the calculator is running with the ***wand send*** command. This send a code (entered in decimal!) as follows

wand send 129 send decimal 129 (0x81) to the Wand emulation, this is the code for the ‘1’ digit entry. Up to 9 codes can be entered on the command line

Not (yet) implemented are any peripheral functions (XROM). Due to limitations in the Wand it is not possible to enter direct synthetic instructions.

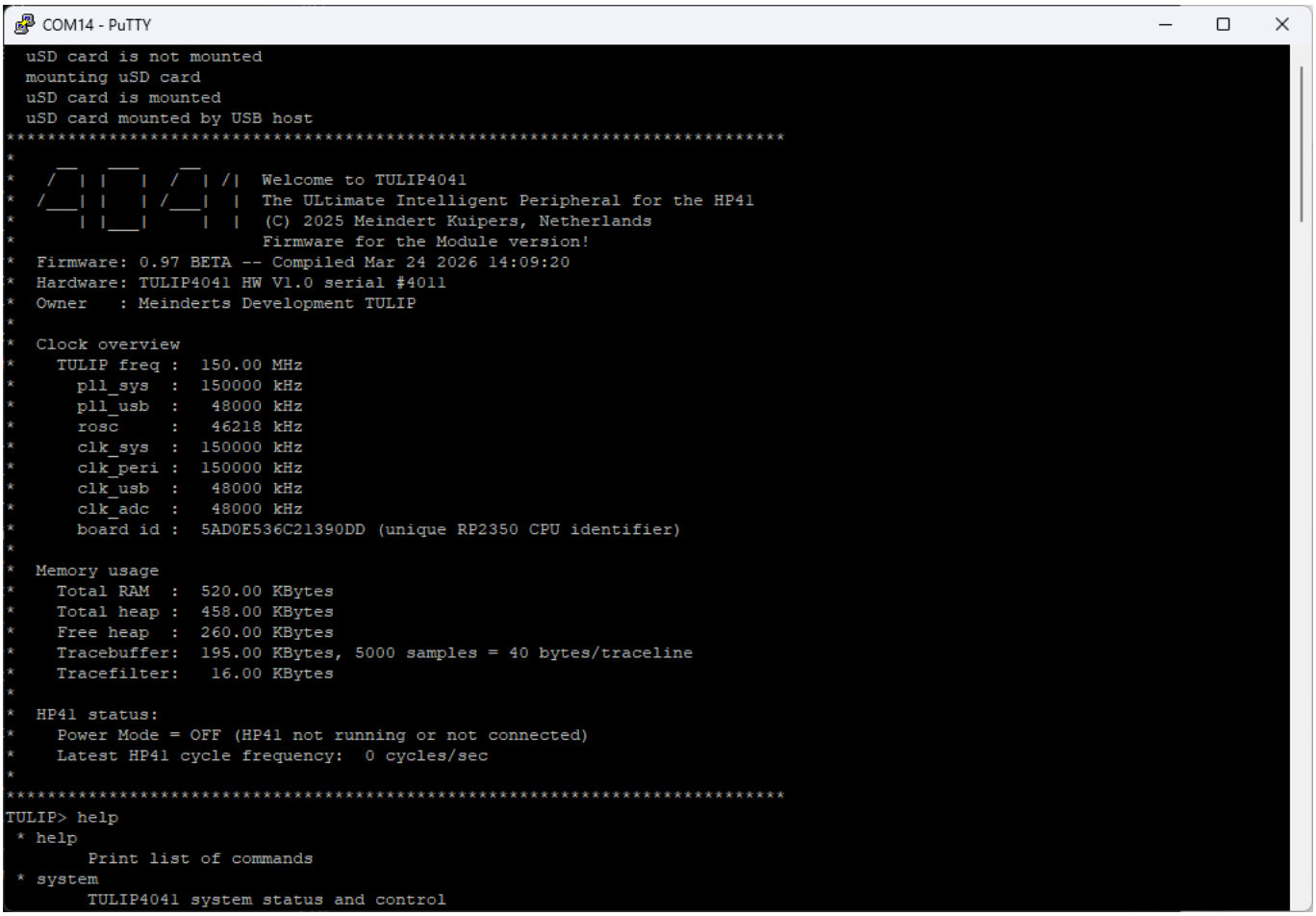
WARNING: it is possible to create an ALPHA string (Wand type 7 or 8) containing characters <0x10. This may lead to a crash of the HP41 (endless loop) due to a possible bug in the Wand ROM. Recovery is possible only by removing the batteries and fully disconnecting the TULIP.

11. TULIP4041 first steps

The instructions below will guide you through the steps to get up and running with the TULIP4041 quickly.

1. Format a micro SD card (minimum 2 GByte) with the exFAT (recommended) or FAT file system
2. Create a repository in a subdirectory on the card of ROM and MOD files that you wish to use in your TULIP (can also be done after step 3). Let's call this myROMrepo
3. Plug the microSD card in your TULIP and connect it with your host PC, verify if the card is visible as a removable drive in your host computer
4. Connect a Terminal Emulator with the TULIP Command Line Interface. Try the newly discovered serial ports until you see the TULIP welcome message. Use the ***sdcard connect*** command in the CLI to connect with the host if the USB drive was not visible. When using the FAT file system the detection by the host can take some time, even up to several minutes when a large capacity SD card is used.
5. Update to the latest firmware (see the instructions in the previous chapter) if needed
6. Type the command ***dir myromrepo*** (do not care about the case of the subdirectory name) and verify if all your files are visible
7. Type the ***list*** command to check which ROMs are already in FLASH. If this is the first time the TULIP is used it may contain the ROM files used for production testing . Go to step 12 in case the list is garbled or if no files are visible at all. If the system is newly initialized it will show only the "TULIP4041 FLASH HEADER" file of type 41.
8. The FRAM file system just be initialized as well. Type the command ***fram INIT*** to do this
9. Type the command ***import myromrepo ALL*** (ALL must be uppercase!). This will copy your ROM repository to FLASH memory which is needed for the ROMs to be pluggable.
10. Use ***list*** again to verify if all your roms are imported
11. Connect the TULIP with your calculator. While doing that check which ports are physically occupied
    You can now use the plug command. Say that you have imported the set of PPC roms, PPCL.rom and PPCU.rom. Assuming that you have Port 4 free (this is Page E and F) you can type (the case of the ROM name and the Page do not matter
    TULIP> ***plug PPCL.rom E***
    TULIP> ***plug PPCU.rom F***
12. Use ***cat*** in the TULIP CLI to confirm that the ROMs are plugged and enter CAT 2 on the HP41 to confirm that the PPC ROM is now plugged in the calculator. The configuration is saved in FRAM and will be available again after a power cycle
13. In case the file system is corrupted you need to fully initialize the FLASH File System with the following steps:
    TULIP> ***flash NUKEALL***
    TULIP> ***flash INIT***
    Or ***fram NUKEALL*** and ***fram INIT*** to do this for the file system in FRAM.
    And go back to step 7

12. TULIP4041 Command Line Interface reference



The TULIP4041 Command Line Interface (CLI) is accessible using the first (usually lowest numbered) virtual serial port. Upon a reboot or reconnect of the unit the CLI will show a welcome message with system information and a prompt. Commands can be issued at the prompt. The CLI buffers the commands which can be recalled and edited with the up- and down-arrow keys. This paragraph gives an overview of the available commands. Details on plugging and unplugging ROM images and other features relevant to HP41 device emulation and tracing are given elsewhere in this document. Many commands have parameters, these are explained with the command. Commands are case sensitive, most commands can be entered in lower case, some critical commands or parameters have to be given in uppercase.

The <TAB> key can be used to complete a command, but only the command, not the arguments. Command history is available with the up and down arrow keys.

Parameters in <> are optional, parameters in [square brackets] must be given.

help <command>

Shows all available commands with a short explanation. Shows the help for a specific command when a parameter is typed, for example ***help system***

system <argument>

<no argument>

Shows the welcome message with the system status, including memory usage.

status
Shows the welcome message (same as the command system without a parameter).

pio
Shows the results of the PIO state machine initialization.

cdc
Shows the status of the virtual serial ports connected to the USB port.

cdcident
Sends a message to each connected virtual serial port to facilitate identification of the connected serial ports. Use only when a terminal emulator is connected to the ports. When a virtual IL device is connected this command may confuse the virtual device.

REBOOT
Resets the TULIP4041 unit. Note that a reboot can take a few seconds before the TULIP4041 is able to respond to the HP41 and before the CLI can be accessed. All virtual serial ports are disconnected, automatic reconnect depends on the host computer and the terminal emulator used. Only works when the HP41 is not running (HP41 PWO line is low) or no calculator is connected.

BOOTSEL
Stops the TULIP4041 firmware and enters RP2350 BOOTSEL mode for example for a firmware upgrade. Only works when the HP41 is not running (HP41 PWO line is low) or no calculator is connected. Will unplug all plugged embedded ROM images.

poweron
Drives ISA for 20 microseconds to switch on the calculator. Works only when PWO is low. Driving ISA causes the HP41 to wakeup and execute the IO Service requests. If there is no active request the calculator will go to STANDBY mode again, this is typically not visible in the calculator display. The tracer will show the instructions executed. This command is equivalent to pushing the button on a Wand for example, except there is no interrupt routine executed.

calcreset
Drives PWO for 10 microseconds to halt the calculator. Works only when PWO is high. This will immediately halt all running code in the calculator, and can be used to recover from an mcode loop or other kind of lock-up. Test this by starting a CAT 3 for example and then type ***system calcreset*** in the CLI while the catalog is running. The CAT listing will immediately halt. Back in the good old days this function was implemented in a small module device or as an addition to an MLDL. Very useful to recover from experimental mcode programming, it was therefore also known as a crash killer. Executing this command may upset the tracer as the synchronization may be lost. On the DevBoard the correct jumper must be set. On the Module version the solder jumper JP2 must be closed (it is normally open).

configinit
Initialized the persistent system configuration in FRAM to its default values. Use only in case the configuration seems to be corrupted.

configlist
Shows the persistent system configuration. A bit cryptical, use only for support purposes.

gpio
Shows the status of all IO of the RP2350 processor, used for troubleshooting

owner <owner string>
show/program the TULIP owner string. The string (must be enclosed in double quotes if it contains spaces) is programmed in a reserved part of FLASH memory. Any string can be programmed op to 80 characters. When a new string is programmed the previous one is always erased. Without argument shows the current value.

serial [4-digit serial number] (TULIP module only)
show/program the TULIP serial number. The serial number is programmed in processors' OTP and can be programmed only once, this is done during production testing of the module version.

debug
Cycle through the inline debug options (only on or off currently). Use only for advanced troubleshooting

sdcard <parameter>

<without parameter>
Shows the uSD card status report and will mount the card

status
Shows the uSD card status report and will mount the card

mount
Mounts the uSD card and prepares for use. The uSD card is automatically mounted upon start of the TULIP. Use only when plugging a uSD card in a powered system

unmount
Unmounts the uSD card and disables the use of it. Use this command prior to removing a drive in a powered system

mounted
Check if the uSD card is mounted

connect
enables use of the uSD card as a USB drive on the host computer when it has a valid USB connection to a host PC. Upon start of the TULIP the uSD card is mounted and connected

eject
disables the used of the uSD card as a USB drive from the host PC. The card is still available to the TULIP until unmounted. Use only after ejecting the TULIP drive on the host computer to prevent corruption of the cards filesystem. On the host the drive itself will still exist after ejecting but cannot be used until it is connected again

rtc <argument>
(Module version only) Used for testing the PCF8523 RTC and backup battery. For development and testing only, use at your own risk

<no argument>
Shows the RTC status and scans the I2C bus for any other connected devices

status
Shows the RTC status and scans the I2C bus for any other connected devices

set YYMMDDHHMMSS
Set the RTC to the given date and time

get YYMMDDHHMMSS
Get the current date and time from the RTC

reset
Reset the RTC

dump
Dump all RTC registers

clear
Clears the CLI console window, typically does not clear the complete scroll buffer. Use a command in your terminal emulator to do that. This may not work on all terminal emulators.

blink <b>
Blinks the LED on the Pico board. Blinks b times, when no value is given will blink 5 times. B must be between 1 and 9. Use a value of 0 to toggle the LED status

printer <argument>
Controls the HP82143A printer emulation. When this ROM is not (virtually) plugged, most actions will be stored and become effective when the ROM is plugged. Most status settings (power, printmode and paper) are saved on powerdown.

<no argument>
Show the printer status including the print mode and details of the status/control register

status
Show the printer status including the print mode and details of the status/control register

output
Cycle through the printer output options: none, serial, infrared or both

serial
Cycle through the printer serial emulation options: pure HP82143A (for use with a simulator program on the host) or ASCII only (and UTF-8 only) for sending non-graphics printer output to a terminal emulator, applies only to the virtual serial port, NOT to the IR output which is always HP82143A compatible and supports graphics. ASCII and UTF-8 serial modes do not support graphics

power

Toggle the printer power mode. The printer must be ON for any printing operations. By default the printer is OFF (like the real one). Once switched on the printer will stay on

mode
Cycle through the printer mode options like the slider on the printer: MAN, TRACE and NORM

paper
Toggle the printer Out Of Paper status. Default is no paper, like in real life you must put paper in for the printer to work.

print
Does a virtual push of the PRINT button. Prints X or ALPHA depending on the calculator mode, or enters PRINT (or PRA) in a program when in program mode.

adv
Does a virtual push of the ADV button. Virtually advances paper in the connected simulator or enters ADV in a program when in program mode

irtest
Test the infrared LED with an IR receiver. Sends a simple string, this is not a printer test string

irtog
Turn the infrared led ON or OFF, used for test and development. Do not run the LED on for longer time as this consumes power

tracer <argument>
Controls the HP41 bus tracer. Without argument will toggle tracer on/off. Commands may be given while the tracer is running. For more detailed information see the chapter about the HP41 Tracer. Pressing a key in the tracer window will toggle the tracer activity. Note that that will most likely lead to an overflow when resuming tracing again. Note that the tracer settings are not always automatically saved, this must be done with the ***tracer save*** command. This is done to allow control of the tracer while the calculator is running, and a running calculator prevents access to FRAM, where the settings are saved.

<no argument>
shows tracer status

status
shows the tracer status

buffer <size>
shows the size of the main tracer buffer in number of samples. The size can be adjusted by giving the required number of samples, and a subsequent reboot is needed for the change to become effective. The size is maximum 10.000 samples (limited by available memory)

pretrig <size>
shows the size of the pre-trigger buffer (not in use yet) and allows the size of the pre-trigger buffer to be defined. Default is 32 samples, maximum is 256 samples

trace
toggle trace enable/disable

hpil
toggle tracing of the HP-IL frames to the ILSCOPE virtual serial port

pilbox
toggle tracing of the PILBox serial frames to the ILSCOPE virtual serial port

ilregs
toggle tracing of the HP-IL frames and registers to the trace window. Disabling (if you are not interested in this) reduces the load on the tracer firmware

save
save the tracer settings in the FRAM persistent storage. This is done to allow control of the tracer while the calculator is running, and a running calculator prevents access to FRAM, where the settings are saved.

mnem
cycle through the tracer disassembler mnemonics options: none, JDA (Jacobs-DeArras) or HP style mnemonics. Selecting none offers the best possible performance and can increase the amount of samples without overflow

filter
toggle filter enable/disable. When the filter is enabled, only samples that pass the filter will be traced, when the filter is disabled, all samples will be traced. The filter settings are configured with the ***filter*** command

filter <argument>
Functions for handling the inline filter of the mcode tracer

<no argument>, status or list
list show the current filters

dump
dump the filter settings for debugging

save [filename.trf]
save the current filter settings to a file on the uSD card

load [filename.trf]
load filter settings from a file on the uSD card the filename can contain an existing subdirectory name existing files WILL be overwritten after asking

filter
toggle filter enable or disable

block
show all filter BLOCK entries

pass
show all filter PASS entries

block all
block ALL trace samples

block pX
add a BLOCK filter for Page x (hex)

block XXXX
add a BLOCK filter for address XXXX (hex)

block XXXX YYYY
add a BLOCK filter for the range XXXX..YYY (hex)

pass all
remove all filters and pass all samples

pass pX
add a PASS filter for Page x (hex)

pass XXXX YYYY
add a PASS filter for the range XXXX..YYY (hex)

umem <argument>
Controls User Memory emulation. Without argument will show the current status (number of modules plugged). Do not use with an HP41CL, as it has its own Extended Memory emulation. Ensure to set the number of modules to 0 when using the HP41CL in combination with the TULIP4041.

<no argument>
shows the plugged Extended Functions/Memory modules

status
shows the plugged Extended Functions/Memory modules

[N]
N=0..4
Plugs N User Memory modules, 4 is the same as a Quad Memory module. Be careful of conflicts with physical memory modules or the HP41CX or -CL

dump
creates a listing of the User Memory contents. Only non-zero registers are shown.

ERASE
clears all emulated User Memory to 0's

xmem <argument>
Controls Extended Memory emulation. Without argument will show the current status (number of modules plugged). Do not use with an HP41CL, as it has its own Extended Memory emulation. Ensure to set the number of modules to 0 when using the HP41CL in combination with the TULIP4041.

<no argument>
shows the plugged Extended Functions/Memory modules

status
shows the plugged Extended Functions/Memory modules

[N] [M]
N=0,1 M=0,1,2
plugs N Extended Functions Memory (0 or 1) and M Extended Memory modules (0, 1 or 2), be careful of conflicts with physical Extended Functions/Memory modules. The Extended Functions ROM image must be plugged separately or plugged physically or already available in an HP41CX

dump
creates a listing of the Extended Memory contents. Only non-zero registers are shown. Use for your own interest and to verify the pattern for testing FRAM. Help umem

ERASE
clears all emulated Extended Memory to 0's

flash <argument>
Functions for managing FLASH memory.

<no argument>
shows the FLASH chip ID and size

status
shows the FLASH chip ID and size

dump <ADDR>

shows a hex dump of the FLASH memory contents for the FLASH File System. Always shows 4K bytes. Use a hex address (offset in the Flash File System) for the start address. Subsequent ***flash dump*** commands without an address will continue the dump

INIT
initializes the Flash File System by creating the header file and start of the chain, FLASH must be fully erased first

NUKEALL
fully erases the complete Flash File System, only the space reserved for this, never the Flash memory used for the firmware

fram <argument>
Functions for managing FRAM memory.

<no argument>
shows the FRAM chip ID and size

status
shows the FRAM chip ID and size

dump <ADDR>
shows a hex dump of the FRAM memory contents for the FRAM File System. Always shows 4K bytes. Use a hex address for the start address. Subsequent ***fram dump*** commands without an address will continue the dump

INIT
initializes the FRAM File System, for now limited to the portion of the ROM map

NUKEALL
fully erases the FRAM to all zero's

dir <subdir>
Shows a directory listing of the microSD card root directory or subdirectory (when given)

list <argument>
Shows files and details of the files in the Flash File System.

<no argument>
Shows a listing of all files with filename, type, size, offset of the file in the Flash File System and the offset to the next file.

ext
show an extended listing of all files including some details of the file content

all
shows all files including deleted files

<filename>

Shows an extended listing of a single file with details of the contents. A partial name can be entered, and then the first matching file will be shown.

flash or **qrom**
Show only files in either FALSH or QROM (FRAM), can be used with the ***ext*** or ***all*** options

import <argument>
Import files from the uSD card into FLASH. Only supported file types can be imported, this is checked by the file extension. File name can be maximum 31 characters (including dot and extension). The filename must include any subdirectories (subdirectory is not counted in the filename length). Use the ***qrom*** argument when importing files in QROM (FRAM), by default files are imported in FLASH. Files with a directory indication are imported from the current directory (use the ***cd*** command to change the current directory). Filenames must be unique in the TULIP file system.

<no argument>
not valid

<filename>
import a single file in FLASH

<directory> ALL
import all files in a directory on the uSD card into FLASH. Only supported filetypes will be imported, existing files will be skipped. Files will be skipped if there is not enough room in the files system

<filename> compare
compare a single file with the one in FLASH or FRAM/QROM. Will tell if the files are identical or not and if the file (if in FLASH) can be updated without prior erasing

<filename> UPDATE
update a single file with the one in FLASH or FRAM/QROM. For a successful update the following must apply:

- The file name is identical
- The file size is identical
- The contents are different
- There is no need to first unplug the ROM image if it was plugged if:
  - The exact same bank assignment is used in case of a multi-bank module
  - The exact same page order is used in case of a multi-page image
  - There are no changes in the hardware support of a MOD file if it is plugged

<directory> compare ALL
compare all files in a directory on the uSD card with the ones in FLASH. Can be used to find out which files need updating. Does not work with files in FRAM/QROM

<directory> UPDATE ALL
update all files in a directory on the uSD card with the ones in FLASH. The same conditions apply as for updating a single file. Does not work with files in FRAM/QROM

[filename] qrom
Import a single file in the QROM (FRAM) file system

export <argument>
Save a file in QROM (FRAM) to the uSD card. Files in FLASH cannot be exported.

files from the uSD card into FLASH. Only supported file types can be imported, this is checked by the file extension. File name can be maximum 31 characters (including dot and extension). The filename must include any subdirectories (subdirectory is not counted in the filename length). Use the ***qrom*** argument when importing files in QROM (FRAM), by default files are imported in FLASH. Files with a directory indication are imported from the current directory (use the ***cd*** command to change the current directory). Filenames must be unique in the TULIP file system.

[filename]
Save the file from QROM to a file with the same name in the current directory. Existing files will never be overwritten

[filename] [<dir/>new filename]
Save the file from QROM to a file with a new name in the current directory, or in the name directory. Existing files will never be overwritten

delete <filename>
Delete a file from FLASH or FRAM memory, and frees the space for new files. The file is marked for deletion only but will not show up with the ***list*** command, unless ***list all*** is used.

cd <directory>
Change the working directory on the uSD card. All file operation son the uSD card will now operate on this directory. The indicated directory must exist, and the TULIP firmware supports only one directory level

<no argument>
Show the current working directory

[directory]
Change the working directory

[.. or /]
Change the working directory to the parent or root directory

plug <argument>
Plug a ROM image (virtually) in the HP41. Supported files are .ROM and .MOD (MOD files only of MOD1 type)

hpil
plugs the embedded HP-IL module in Page 7

ilprinter
plugs the embedded HP-IL printer module in Page 6. The module can only be used when the HP-IL module is plugged

printer
plugs the embedded HP82143A printer module in Page 6

filename.rom P
plugs the file “filename.rom” in Page P (P in hex) if that Page is available. The file is always plugged in Bank 1

filename.rom P B
plugs the file “filename.rom” in Page P (P in hex) and Bank B (B=1..4) if that Page and Bank is available

filename.mod
plugs the file “filename.mod” according to the Page/Bank settings in the MOD file and enables the hardware (if supported). Will automatically find the best matching slot. If the MOD file contains a mix of supported and not supported ROM images the supported images will be plugged

filename.rom <T>
By omitting the Page will plug the file “filename.rom” in the first available Page (always Bank 1) starting at Page 8. When a T (for Test) is added the selected Page will be reported but the file will not be plugged. When used with MOD files the images will be plugged

unplug <argument>
Removes a ROM image from the HP41 (only images plugged in the TULIP4041)

P
Unplug the ROM in Page P (P in hex), does not apply to reserved Pages

P B
Unplug the ROM in Page P (P in hex) and Bank B

all
Unplug all plugged ROMs, except reserved Pages

ALL
Unplug all plugged ROMs including all reserved Pages

reserve <argument>
Reserve a Page for a physical module to assist the Plug function

P <comment>
Reserve Page P (P in hex), use the comment for your own use. Comments must be enclosed in parenthesis if it contains spaces

cx
Reserve Page 3 and 5 for the CX ROMS (Timer).Is effectively the same as reserve timer as Page 3 is always reserved and no ROM images can be plugged there

timer
Reserve Page 5 for the TIME module. Has the same effect as ***plug cx***

printer
Reserve Page 6 for a Printer (HP82143, IL Printer or IR Printer)

hpil
Reserve Page 7 for the HP-IL module

clear [P]
Cancel the reservation for Page P (in hex)

clear all
Cancel all reservations

cat <argument>
Show the plugged and reserved ROM images

<no argument>
Show a listing of all plugged and reserved ROM images

P <B>
Show the details of the ROM image (with a hex dump) plugged in the given Page, optionally the Bank can be given. Page number must be in hex

emulate <argument>
Enable or disables emulated hardware, toggles the emulation status. Emulations of HEPAX RAM and QROM are handled by the hepram and qrom commands.

<no argument>
Show the status of the emulated hardware

status
Show the status of the emulated hardware

hpil
toggle HPIL hardware emulation, use when plugging the HP-IL module as a .ROM file instead of the embedded ROM

printer
toggle HP82143A printer emulation

wand
toggle HP82153A WAND emulation

zeprom P
toggle ZEPROM emulation in Page P (hex) for sticky bankswitching, P is the Page number in hex (0..F). This will apply for both the odd and even Page in the Port. The ZEPROM itself does not need to be plugged

qrom <argument>
Manage the files plugged from QROM (FRAM)

<no argument>
Show the QROM status by show all QROM (and HEPRAM) pages

status
Show the QROM status by show all QROM (and HEPRAM) pages

enable p

Toggle the read enable of Page p (in hex), this also applies to non-QROM Pages in case you want to temporarily disable the access to a Page. Write access is still enabled

qrom p
Toggle the QROM status of a Page, this allows enabling QROM functions (WROM specifically) to a Page that was not marked as RAM in the MOD file.

wprot p
Toggle the write protection of Page p (in hex), this applies to all banks in the Page.

hepram p
Toggle the HEPRAM status Page p (in hex), this effectively enables a Page to be recognized by the HEPAX ROM by allowing the WPTOG instruction in the Page.

polling p
Toggle the read enable of the polling entries (address xFF4..xFFA) of Page p (in hex), this can be useful when developing an mcode ROM. This also applies to FLASH Pages in case you want to temporarily disable the polling entries.

CLEAR p <b>
Completely erase Page p (in hex) and bank b to all zeroes

hepram <argument>
Manage the files plugged as HEPAX RAM Pages. Please refer to the excellent HEPAX manual for background information about these functions and especially the HEPAX RAM chain.

<no argument>
Show the HEPAX Ram status by show all HEPRAM (and QROM) pages

status
Show the HEPAX Ram status by show all HEPRAM (and QROM) pages

ramtog p
Toggle the write protection of HEPRAM Page p (in hex), like the HEAPX RAMTOG function. Only works on HEPAX RAM Pages. Do not write protect a Page that is active in the HEPAX RAM chain

reserve p
Reserve Page p (in hex) so it is not used in the HEPRAM chain. This function writes the value 0x000 to address xFF3 in the Page (bank 1)

release p
Release the reservation of Page p (in hex) so it can be included in the HEPRAM chain. This function writes the value 0x100 to address xFF3 in the Page (bank 1)

CLRAM p
Clear HEPRAM Page p, just like the HEPAX CLRAM function. Will respect the write protect status of a Page.

INIT
Initialize the HEPAX RAM chain, must be used to allow access to the HEPAX file system by the HP41 HEPAX functions. Will build the chain only with Pages that are specifically marked as HEPRAM and that are not reserved (see the ***hepram reserve*** and ***hepram release*** commands). HEPAX RAM will not be erased, to

allow changes in the HEPRAM configuration while keeping the ROM contents. Will respect the write protect status of a Page.

INITALL
Initialize the HEPAX RAM chain, must be used to allow access to the HEPAX file system by the HP41 HEPAX functions. Will build the chain with ALL Pages that are marked as HEPRAM including those that are reserved. HEPAX RAM will not be erased, to allow changes in the HEPRAM configuration while keeping the ROM contents. Will respect the write protect status of a Page.

wand <argument>
Functions for controlling the wand emulation, the WAND ROM must be plugged in the TULIP

<no argument>
Shows the WAND emulation status

status
Shows the WAND emulation status

scan [filename]
Scans the file [filename], must be in the .WND format generated with the hp41uc utility with the command line hp41uc /r=infile<.raw> /w<=outfile<.wnd>> <"Program Title">. This is human readable text file on the uSD card that is interpreted by the TULIP firmware and sent to the Wand emulation layer

list
Show all HP41 functions supported by the ***w*** command

test
Sends the BEEP function to the Wand emulation for testing

send
Sends up to 8 decimals (decimals 0..255) to the Wand emulation as a barcode, for example:
wand send 224 134 (send BEEP function)

WARNING: sending characters <0x10 in a string when NOT in PRGM mode may lead to a crash of the HP41! This is a likely bug in the Wand ROM.

w <argument list>
Implementation of the Wand Paper Keyboard. The <argument list> may contain a list of HP41 functions, strings, numbers or individual keys, case insensitive. Arguments are separated by spaces. When the HP41 is in PRGM mode the argument list will be entered as program lines

[HP41 Function]
One or more of the functions as shown in the wand list command (case insensitive)
w *pi sin x^2 sto 00 prgm gto . .*

[number]
Individual digits or a complete number, the entries are interpreted as individual key presses:
-123.56E12

[ALPHA string]

Enclosed by double quotes generates a single ALPHA string with maximum 14 characters. Characters 0..15 are replaced by spaces as these are incorrectly handled by the Wand ROM and can lead to HP41 crashes. Some escape characters are supported after the '\' (backslash) character. Characters >127 are supported

w "abc" simple string

w "\+ABC" \+ is for append

[single ALPHA characters]

When preceded by a single quote the characters are sent as individual ALPHA key presses, only characters <128 are supported. A single quote without characters. The string is terminated by a space and does not have to be terminated by the quote. When there is a quote this will end ALPHA entry mode

w 'ABC' final quote will close ALPHA

w '\+abc escape characters are supported, \+ is for append, ALPHA entry still possible

Escape code	**Result**	**Hex code**
\XX	Hex code XX	0xXX *
\\	\	0x5C
\S	sigma sign	0x7E
\+	append	0x7F
\t	hanging t	0x60
\#	not equal	0x1D
\s	space	0x20
\u	greek mu, or micro	0x0C **
\a	angle symbol	0x0D **
\'	single quote	0x27
\"	double quote	0x22 **

* Characters 0x00..0x0F are replaced by a space in strings with the double quote "

** not supported in ALPHA strings (starting with ")

15. Limitations

- HP-IL and HP-IL printer emulation with PILBox emulation
    - RFC and CMD frames are not processed in the same way as in the hardware PILBox. This has no functional impact, the ILSCOPE window in the virtual HP-IL device on the host will be slightly different with too many RFC frames

All issues are maintained in the TULIP github.

16. TULIP4041 Hardware: Module version

The TULIP4041 module is a custom board that will fit in an HP41 long sized module (like the HP41 IL module). Its features are nearly identical to the DevBoard version, with a few different pin assignments. Added to the Module version is an RTC with battery backup for emulation of the TIME module). The only limitation of the Module version is that it cannot trace the FI signal from the HP41. The FI signal shown is limited to the FI signal being driven by the TULIP4041.
The TULIP module has 4 Mbyte of FLASH memory in an SOIC-8 package that is hand solderable. It can be replaced by a larger sized device if needed (requires recompilation of the firmware). A number of signals are available on holes for a 2mm header.
The TULIP4041 Module is fully assembled and tested and contains the TULIP firmware. It comes with a 3D printed housing. Depending on the chosen option it comes with or without a module connector.

TULIP module features:

- Comes with a 3D printed long module housing that is easy to open and close
- 4-layer PCB
- USB-C connector
- IO signals and CPU debug signals available on 2mm holes (no header provided)
- PCF8523 RTC (no battery provided) with backup battery carrier
- 4 Mbyte of FLASH and 256 Kbyte FRAM
- Micro SD card holder (no memory card provided)
- Signaling LED
- Infrared LED

16.1. The TULIP housing

The TULIP housing is very easy to open and close. Simply take the housing with your thumbs on the connector side and gently push the two halves apart. To close fit the two halves together and gently push the two halves until they snap in place. The TULIP board sits loose in the housing, take care that it does not fall off. When opening the housing with the battery be very careful with the battery wires.
To insert or remove the micro SD card it may be easier to open the housing or use tweezers.

WARNING: do not use too much pressure on the top of the housing. When pushing on a part that does not have support under it the housing may crack.

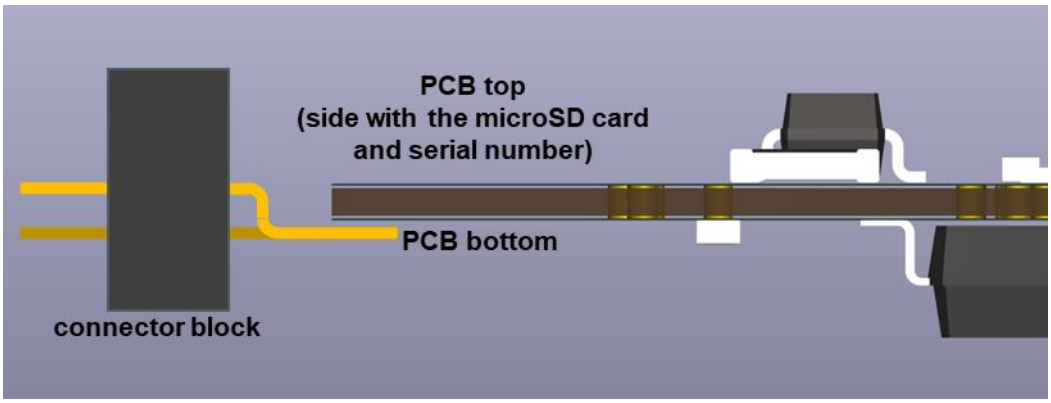
16.2. Soldering the connector

If you have ordered the TULIP module without connector, you must solder one yourself. Please follow the next steps. The connector solder pads are on both sides of the TULIP PCB. This is to allow easier testing in production and to prepare for a possible new connector solution. When using an connector harvested from a real module you must be aware that this must be soldered **on the bottom side of the PCB**!

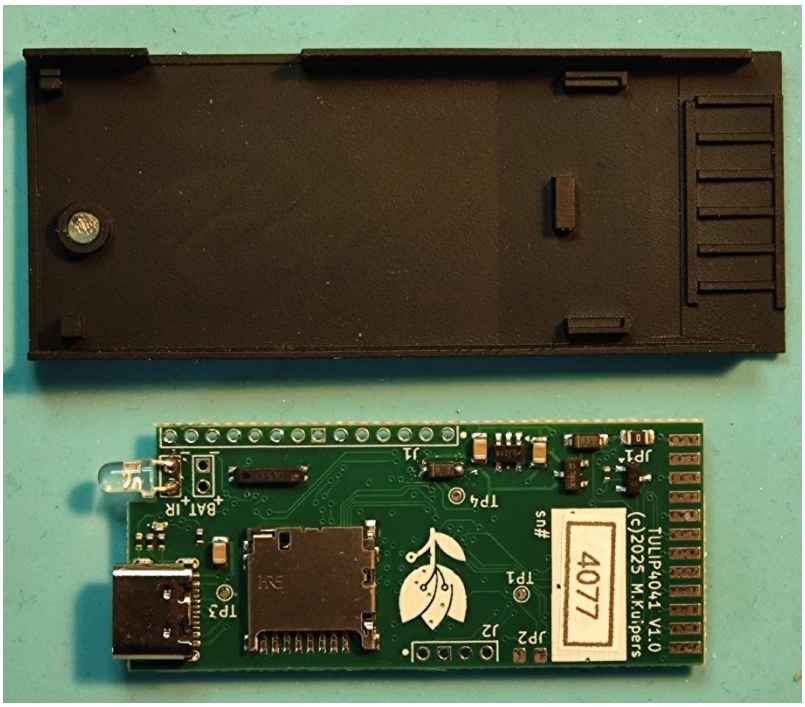
Before attempting to solder the connector ensure that the TULIP Module is functional and that you can access the CLI, microSD Card, FRAM and FLASH memory, just to be certain. When you have the TULIP module without connector you need to harvest one from an older module. Be aware that the standard module connector is soldered on the **bottom** side of the TULIP PCB (the side with most components). It is important that the connector is well aligned and straight with the PCB. The top of the flat housing (the one without the battery holder) is designed to function as a jig.

To achieve this the best is to solder only one pin, and check alignment, reheat and position that pin as needed until the alignment is good. Unsolder a connector from an original module, remove any remaining solder

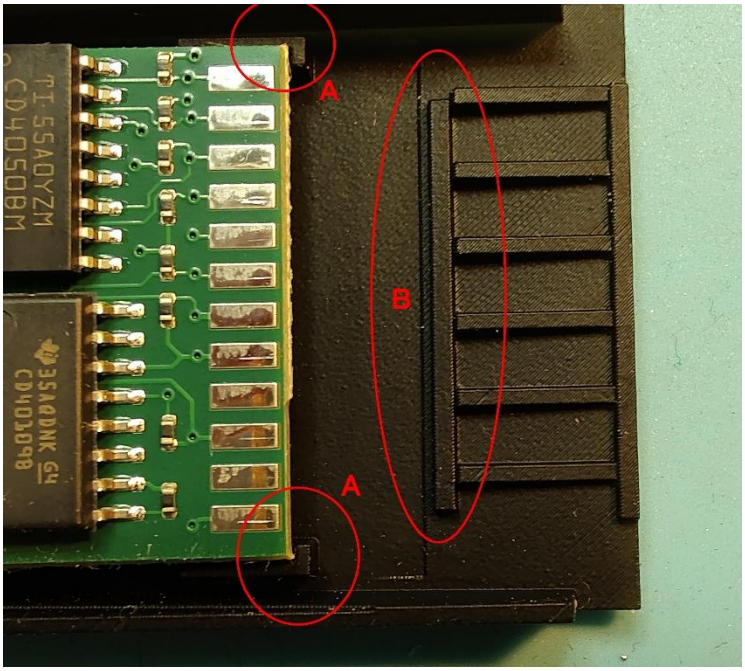
- Remove the uSD card from the TULIP, just in case
- The HP41 connector is mounted on the BOTTOM side of the module, and that is the side where the level shifters are. The TOP is the side where the micro SD card holder is. See the picture below:



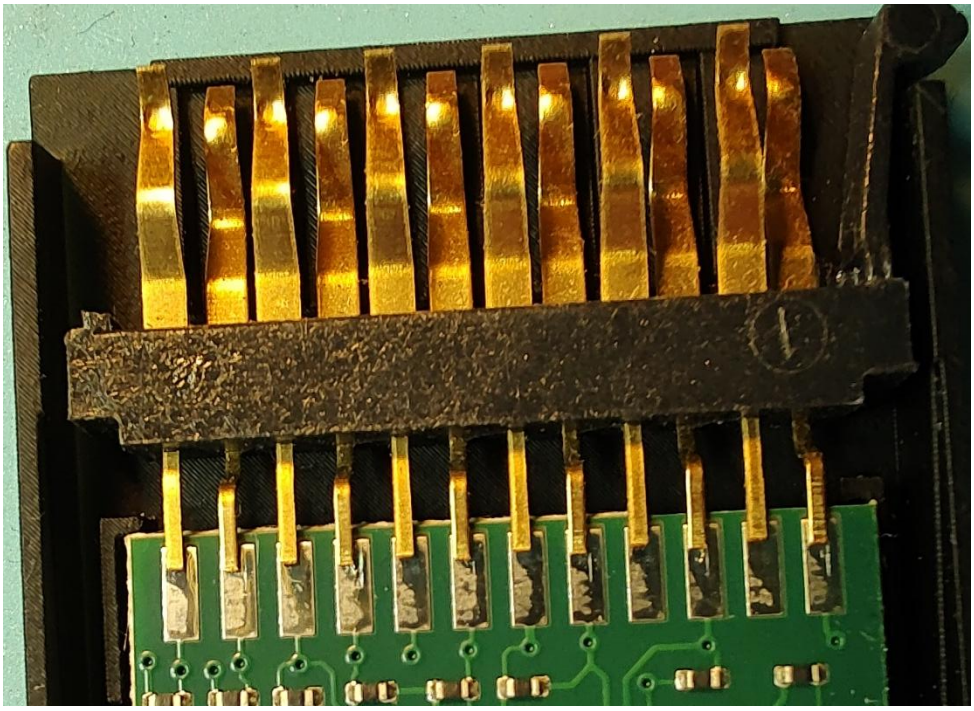
- Put the flat top housing (the one without the battery space) with the open side pointing above on your workspace



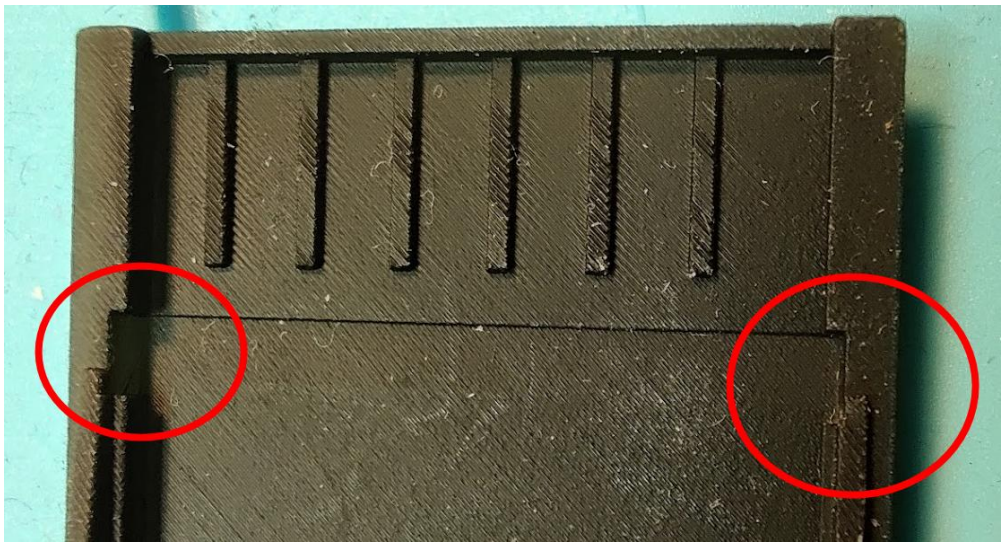
- Lay the TULIP on the housing, aligning with the posts A in the housing (see picture). The PCB side with the 2 level shifters must be pointing towards you. Do NOT solder the connector on the side with the serial number!



- Align the plastic connector body with the calculator connector end with the ridge B on the top housing.
- Position the connector with the solder tabs on top of the PCB. The connectors solder tabs should be flat on the PCB. The connector pins do NOT fully extend on the PCB solder pads, see the picture below



- Carefully solder one pin on the outer left or right side of the connector and verify alignment, then solder the pin on the opposite end. You can put the combination in the bottom housing to check before soldering the other pins. Carefully insert the infrared LED first in the hole, then push the connector body carefully in the slots in the housing. If you are satisfied then solder the other pins. Visually verify the soldering joints and check if the TULIP and connector fits in the housing and all pins are aligned.



SLOTS FOR CONNECTOR BODY ON THE BOTTOM HOUSING

To test the connector:

- Ensure that there are no virtual ROM images plugged in the TULIP and remove the uSD card
- Remove the battery from your calculator and remove the USB cable from the TULIP
- Carefully insert the TULIP in your calculator, do not force it in. It should slide in the port and connect like a normal module. If you sense that more force is needed for insertion then try another port and visually check the pins in the module
- Now insert a battery in the calculator and check if the calculator is running normally. If you can measure the power consumption check if the current does not exceed 20-25 mA
- You may now insert a uSD card in the TULIP and connect with a host computer using USB and open a terminal window to the tracer
- Switch the calculator on again and verify that there is activity in the tracer window. This means that the input to the TULIP4041 from the HP41 is working
- In the CLI type the command ***plug hpil***. This will plug the HP-IL module (but not yet the HP-IL printer ROM). Now switch the calculator on. In the trace windows you should see activity, and occasionally the FI lines in use. Do a CAT 2 on the calculator to verify that the HP-IL module is now plugged.
- For the next tests start the pyIlPer on your host computer, a message in the CLI shows if the virtual devices are connected. Best use a fully installed pyILPer including a virtual drive
- In the HP41 you can now key in the DIR command. This should return a listing of the files on the virtual drive. If this does not happen first do a reboot of the TULIP system by typing the command system REBOOT, and try again.
- Last test is the ISA power on feature. Make certain that your calculator is not running. It may be powered off or in standby mode. Also have the tracer window standby and keep an eye on it. In the

CLI give the command ***system poweron***, and you should see activity in the tracer (but no activity on the calculator). Be aware that this will not switch on the calculator when it was off!

16.3. Installing the battery board

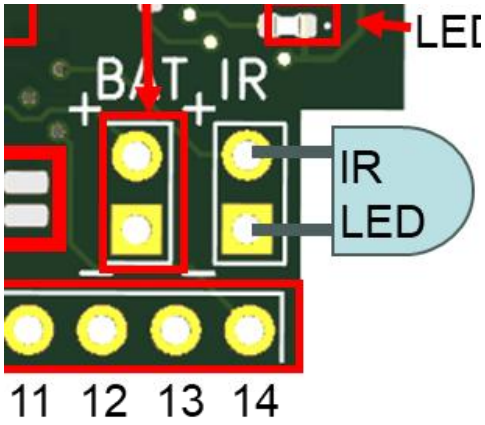
The battery board is a very simply PCB with soldering pads and a battery clip for a CR1620 type lithium coin cell. The battery ensures that the PCF RTC (Real Time Clock) keeps its time when the TULIP is not powered. The RTC is used only for emulation of the TIME module and this functionality is not yet available. This emulation will work fine without the battery, only the current time will not be updated. The battery board must be connected with wires (used very thin wires!) to the holes marked BAT (+ and – are indicated) on the main TULIP PCB. Using the battery board is optional.

16.3.1. Installing the battery board on V1.0 hardware

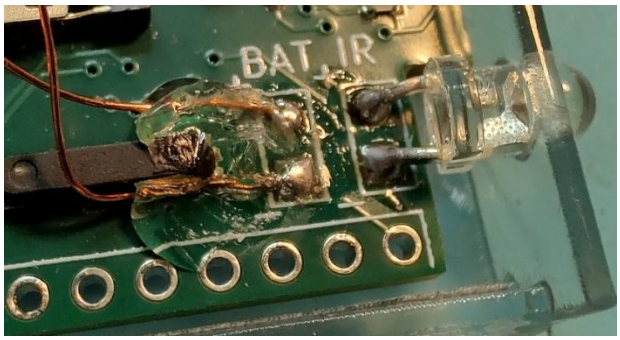
The battery board fits in the space in the special top housing with a bulb. The fit is very tight and it is recommended to first test this before soldering the wires to the board. Inserting it may widen the space a bit.

Instructions for connecting the battery board:

- Do NOT insert a battery in the battery board to prevent a short while soldering
- Solder insulated wires to the pads on the battery board, and note the polarity. Use very thin wire since there is very little space for the wires
- The orientation of the board is not relevant, it fits in two ways in the top housing. Choose an orientation that is convenient to guide the wires to the designated position on the TULIP PCB
- Keep a bit of length on the wires to maneuver and install the battery board in the top housing
- Solder the wires to the terminal on the TULIP PCB marked BAT near the infrared LED and observe the correct polarity



- The soldering joints can be fragile, best is to fix these with a bit of hot glue and guide the wires along the RTC crystal



- Visually verify the soldering joints and insert a battery
- Use the CLI to set the RTC and test:
  - ***rtc set yymmddhhmmss*** (year-month-day-hours-minutes-seconds)

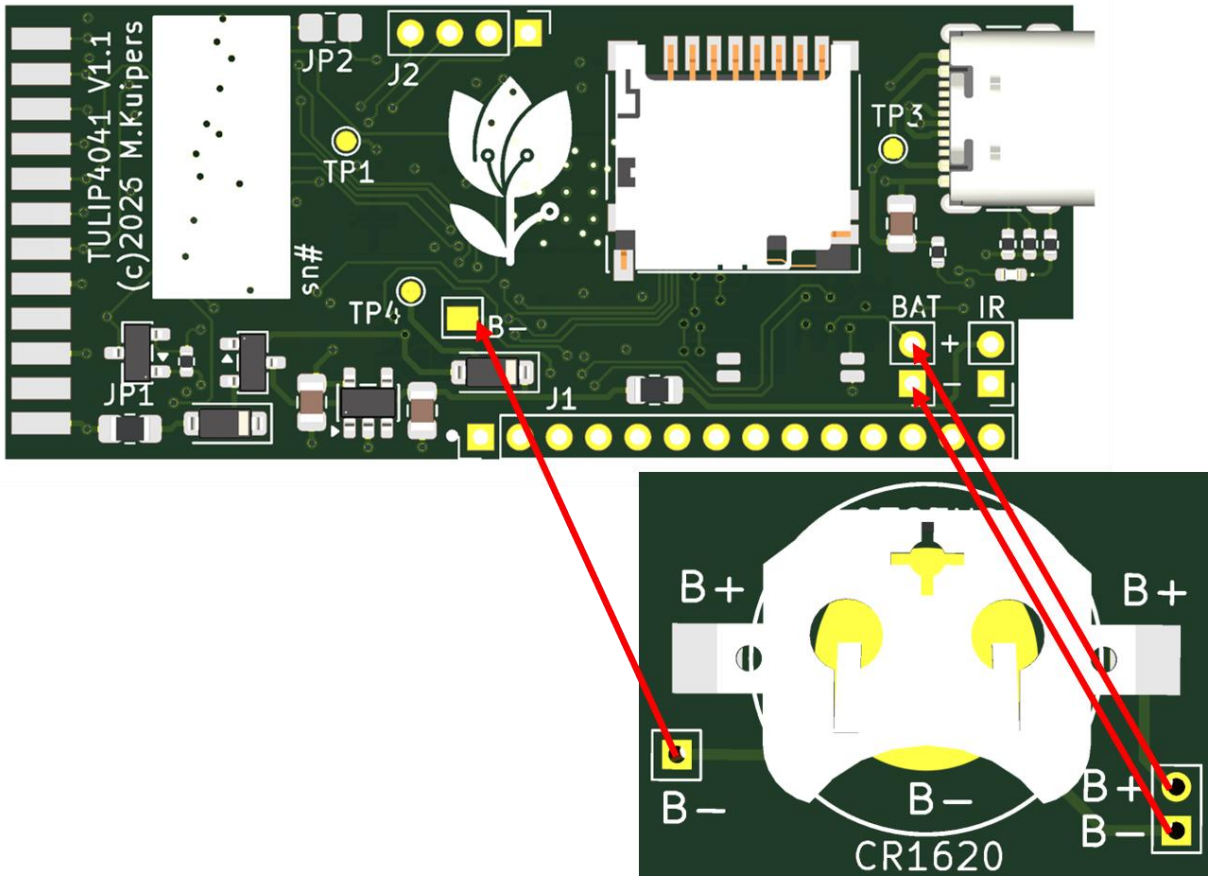
- ***rtc get*** will show the status and report the set time, if the oscillator is running and if the battery voltage is good
- remove power from the TULIP, reconnect and check if the time is still valid.

- Until the TIME module emulation is available I recommend to remove the battery
- Install the battery board in the top housing and carefully close the housing. Especially be very cautious not to squeeze the wires between the battery PCB and the housing as these can be cut (I learned the hard way)

The battery board has a tight fit in the housing and installing it may be a bit fiddly.

16.3.2. Installing the battery board on V1.1 hardware

The TULIP Version 1.1 Module has a new battery board design.



The battery board is mounted directly on the main TULIP PCB. You will need 3 short bstripped wires (about 10mm will do). Please follow the instructions below:

- Do NOT insert a battery in the battery board to prevent a short while soldering
- First solder two wires in a 90 degree angle to the main PCB in the holes BAT+ and BAT-
- Create a small 90 degree hook on the third wire and solder it straight up to the solder pad marked B-, close to TP4. Align the wire as close to the center of the pad as possible
- Carefully slide the battery board over the 3 wires. The wires should easily fit. Especially be careful with the wire on the solder pad B- as the pad may tear off if there is too much later pressure
- Slide the battery board until it touches the micro SD card holder and it is parallel to the main TULIP PCB

- Cut the wires short on the side of the battery PCB and solder
- Remove any excess wire on the bottom side of the TULIP PCB
- Visually inspect the soldering

You may now insert a CR1620 battery, be careful not to create too much lateral pressure, hold the battery holder. Also do this when removing a battery.

- Use the CLI to set the RTC and test:
  - ***rtc set yymmddhhmmss*** (year-month-day-hours-minutes-seconds)
  - ***rtc get*** will show the status and report the set time, if the oscillator is running and if the battery voltage is good
  - remove power from the TULIP, reconnect and check if the time is still valid.
- Until the TIME module emulation is available I recommend to remove the battery
- Carefully fit the top housing (the one with the blob for the battery)

16.4. Preparing for first use

Before the first use of the TULIP module you must consider the following:

- To load virtual modules you must use a micro SD card unless you only want to use HP-IL or the Printer. Please follow the instructions to prepare a micro SD card below
- You will need a USB data cable with a USB-C connection on the TULIP end. The other end obviously should match you host computer
- Did you order the TULIP with an HP41 connector? If yes you can skip the paragraph about soldering the connector
- Do you intend to use the RTC with a backup battery? If yes follow the instructions. Please note that the current software release does not support emulation of the TIME module and the RTC cannot be used from the HP41
- In case you plan to use the IO breakout or debug breakout please observe the pinout. The holes for both debug and IO have a 2mm pitch

The TULIP Module is fully tested before shipping. There is no need to test it like the Devboard. In case you are interested, here are the steps used to for testing after production (see also the videos on my youtube channel):

- General visual inspection of soldering joints for complete soldering and soldering bridges
- Apply USB-C connection, this will show the RP2350 drive
- Verify power consumption with the USB tester, measure 3.3V and 1.1V test points
- Insert uSD card with test ROM images, load TULIP firmware
- After reset check presence of drive and 5 COM ports, blue LED should flash twice
- Connect to the CLI and check the following:
  - FLASH serial number, FLASH dump
  - Initialize FLASH with File System Header
  - FRAM check and dump, must have values from initialization
  - Run ***dir*** command to check uSD card operation
  - Mass Import ROM images from test directory, verify if programmed with ***list*** command
  - Check presence of RTC with the ***rtc*** command and ***rtc dump***
- Plug the HP-IL embedded module and verify plugging with ***cat***

    - Remove USB-C and solder infrared LED with the TULIP in bottom case for proper alignment, the legs of the LED must be cut and bent before soldering
- Program the device serial number with the ***system serial 4XXX*** command
- Prepare HP41C test and TULIP test jig with external power to the calculator and multimeter to check current to the calculator
- Plug TULIP in the test jig without USB-C connected and power calculator, monitor power consumption. current to the calculator about 18..20mA when not running.
- Type CAT on the calculator, should show the HP-IL module and type DIR, should show “NO DRIVE”
- Plug USB-C and connect the CLI and tracer ports to a terminal window. Connect pyILPer to the HP-IL virtual port, calculator current should be around 4mA when running
- Type DIR on the calculator and observe the tracer activity and the FI line and correct display of the virtual drive contents
- Type ***system poweron*** in the CLI and observe activity in the tracer
- Prepare the IR receiver and connect with the HP82240 Printer Simulator, and type ***printer irtest***, check the output in the printer simulator window
- Type ***printer irtog*** and observe power consumption for excessive power, type the command again to turn the LED off
- Remove the TULIP from the test jig and unplug USB-C
- If ordered, solder the HP41 module connector using the top housing as a jig
- Plug the TULIP with soldered module (in the bottom housing) in the calculator and repeat the HP-IL tests
- If ordered solder wires (already cut to length) to the battery board, and solder the wires to the TULIP. TULIP must not be powered!
- Cover the wires on the TULIP with a drop of hot glue
- Insert the test battery and USB-C and verify with the ***rtc get*** command if the battery is registered. Set the clock, unplug USB-C and plug again and check the time. Remove the battery
- Attach the labels for the serial number and battery
- Remove the uSD card
- Assemble the PCB and housing

16.5. The micro SD card

Prepare a micro SD card formatted with the exFAT (preferred) or FAT file system on your host computer. Any size larger than 2 GByte will normally work fine. Sizes under 2 GByte will most likely not work as the firmware does not support standard density cards. Keep in mind that high-speed cards may consume more power, and the interface to the micro SD card is not built for speed. Some high-speed high capacity cards may not work at all in the TULIP

Create a subdirectory and copy your favorite ROM images as ROM and/or MOD files here. In the examples we will use a subdirectory *myROMrepo*. Also create a RAW directory for later use. The files may also be copied when the card is in the TULIP module.

- Remove (in a safe way) the micro SD card from the host computer and plug it in the slot of the TULIP module. The Card Detect signal is not used
- When powered and connected to a host PC with USB the SD card will be automatically recognized like a normal USB thumb drive. In some cases it may take several seconds until the card is recognized
- Using the user interface, verify with the command ***sdcard*** if the card is properly mounted by the firmware

- Unplugging the SD card in the TULIP is best be done when the module is powered down, or after unmounting and ejecting the card. The same applies for plugging a card, after plugging use the *mount* and *connect* commands

16.6. Other tests

Furter testing is not really needed, but some tests could be useful.

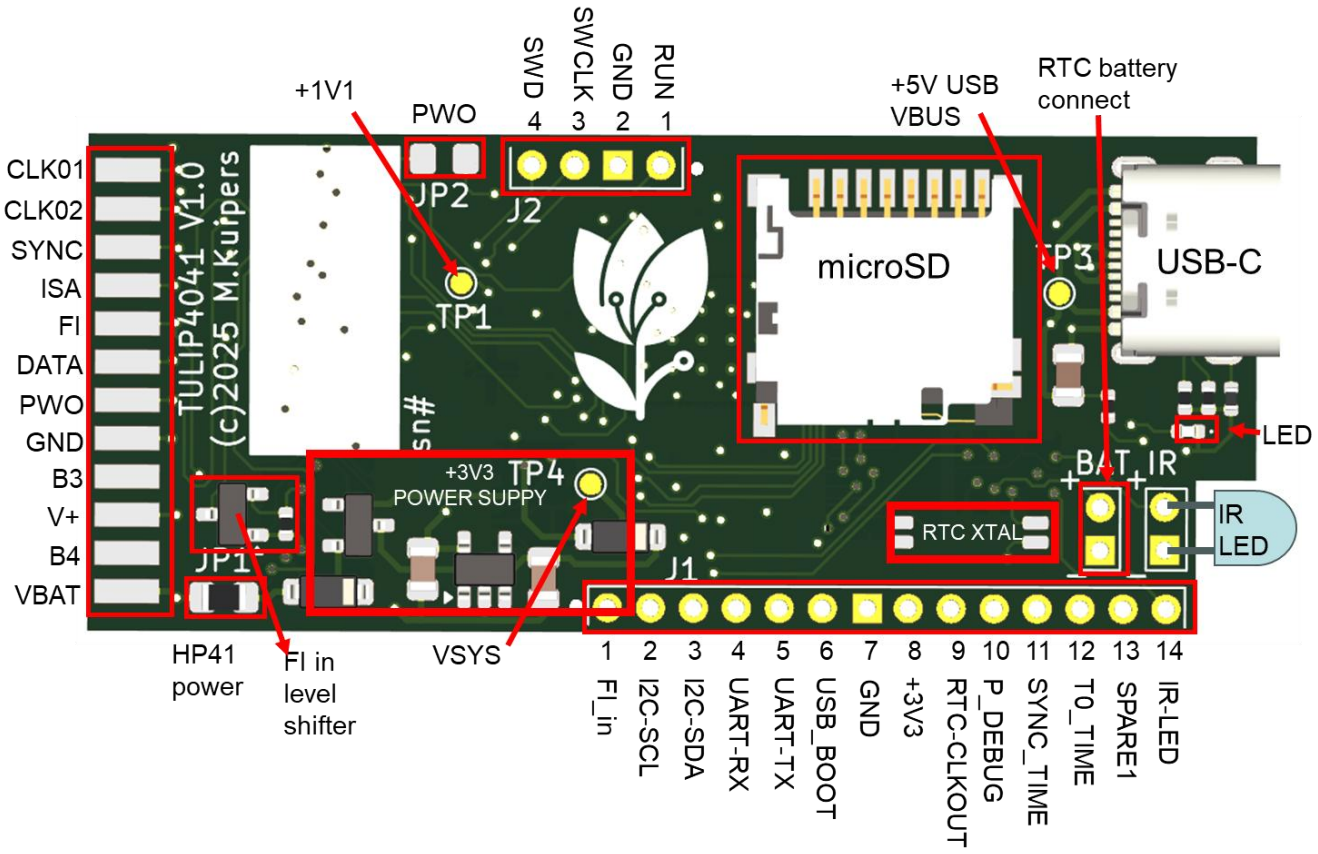
- Test the IR LED: command ***printer irtest*** to send a test string to the IR LED
- Test the signaling LED: command ***blink 8*** to blink the LED 8 times
- Confirm presence of all serial ports on the host computer
- Verify FRAM operation (see the test of the DevBoard)
- Verify the RTC and backup battery operation with the ***rtc*** commands

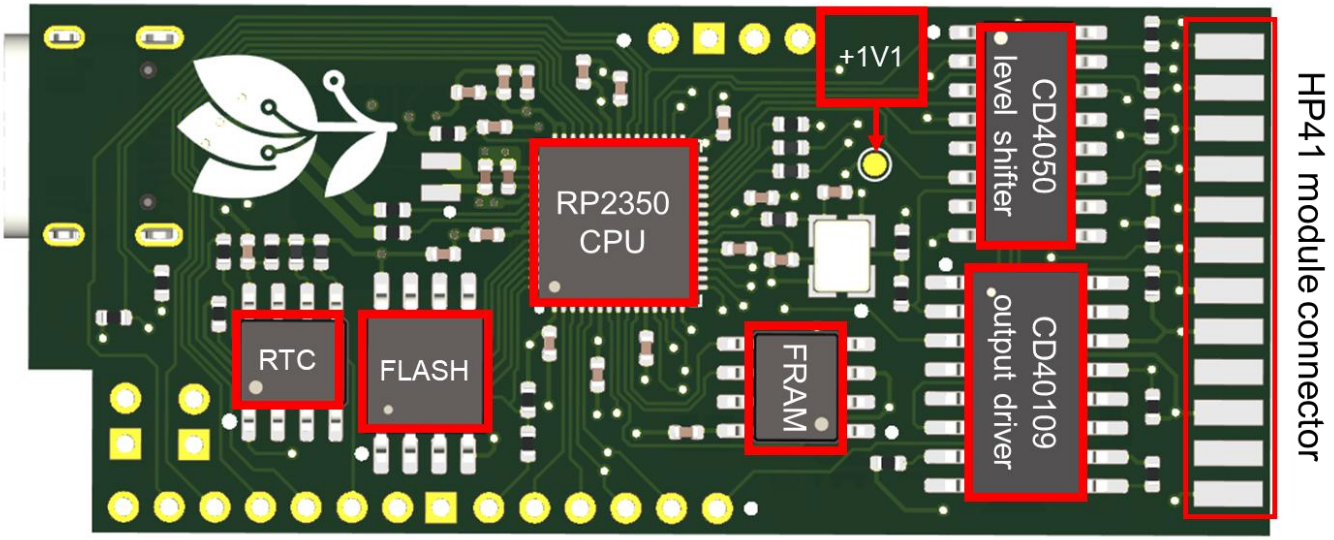
Prepare a micro SD card formatted with the exFAT (preferred) or FAT file system on your host computer. Any size larger than 2 GByte will normally work fine. Sizes under 2 GByte will most likely not work as the firmware does not support standard density cards. Keep in mind that high-speed cards may consume more power, and the interface to the micro SD card is not built for speed.

16.7. TULIP Module IO connections

The TULIP module has the following connections:

- HP41 Module connector
  - The signals of the HP41 bus. The B3 and B4 signals for port identification are not connected
  - The TULIP uses power straight from the HP41 battery when it is not powered by USB. The power consumption is about 20mA (when not using the uSD card) which is a significant load and the batteries will drain more quickly
- USB-C connector
  - For the mass storage (uSD card) and s virtual serial ports
  - TULIP uses power from USB even when the HP41 is connected
- SWD Debug interface
  - This is a 4-pin pad intended for a 2mm header or direct soldering
  - Offers the SWDIO and SWCLK signals plus GND for connection a host computer with one of the available debug options for the CPU to allow low level debugging of firmware
  - The RUN signal is available. Connecting this to GND will rest the CPU
  - Pin 1 is indicated by a white dot on the PCB silkscreen
- IO Breakout
  - This is a 14-pin pad intended for a 2mm header or direct soldering
  - A collection of IO signals for possible extension and/or debugging
  - A pin next to the GND pin is the USB_BOOT and has the same function as the pushbutton on the PICO board to force the processor into BOOTSEL mode
  - Pin 1 is indicated by a white dot on the PCB silkscreen
- There is a blue signaling LED on the PCB that can be controlled by firmware, and it is connected to GPIO25, the same pin as on the PICO development board
- Two holes are used to connect the infrared LED
- Two holes on the PCB are used for the RTC backup battery


CLK01
CLK02
SYNC
ISA
FI
DATA
PWO
GND
B3
V+
B4
VBAT
TULIP4041 V1.0
(c)2025 M.Kuipers
sn#
+1V1
PWO
SWD 4
SWCLK 3
GND 2
RUN 1
JP2
J2
TP1
+5V USB
VBUS
RTC battery
connect
microSD
TP3
USB-C
LED
BAT+
IR+
IR
LED
+3V3
POWER SUPPY
TP4
RTC XTAL
JP1
J1
HP41
power
FI in
level
shifter
VSYS
1 FI_in
2 I2C-SCL
3 I2C-SDA
4 UART-RX
5 UART-TX
6 USB_BOOT
7 GND
8 +3V3
9 RTC-CLKOUT
10 P_DEBUG
11 SYNC_TIME
12 T0_TIME
13 SPARE1
14 IR-LED


+1V1
RP2350
CPU
CD4050
level shifter
CD40109
output driver
RTC
FLASH
FRAM
HP41 module connector

HP41 Module Connector		Module version (RP2350 pinout)
CLK01	input	
CLK02	input	
SYNC	input	
ISA	input/output	
FI	input/output	
DATA	input/output	
PWO	input/output	Optionally driven by TULIP, JP2 must be closed
GND	HP41 GND	
B3	not connected	
V+	HP41 regulated power ~6V	
B4	not connected	
VBAT	HP41 battery	Power for TULIP when no USB connected
Testpoints		
TP1 (top), TP2 (bottom)	+1.1V	From internal CPU regulator
TP3	VBUS, USB +5V	
TP4	VSYS	USB or VBAT power after FET
J2 DEBUG		
1 - RUN	CPU RUN signal	Short to GND to reset CPU
2 - GND	System GND	
3 - SWCLK	Debugger CLK	Use with PicoProbe
4 - SWD	Debugger Data	Use with PicoProbe
J1 GPIO		
1 - FI in	FI input from FI level shifter	Not connected to CPU GPIO, possibly connect to pin13 for future implementation
2 - I2C SCL	GPIO3	I2C connected to RTC, available for other peripherals
3 - I2C SDA	GPIO2	I2C connected to RTC, available for other peripherals
4 - UART RX	GPIO1	UART used by firmware, future other use
5 - UART TX	GPIO0	UART used by firmware, future other use
6 - USB_BOOT	USB-BOOT	Short to GND for BOOTSEL mode while connecting USB
7 - GND	System GND	
8 - +3V3	System +3V3	
9 - RTC-CLKOUT	CLK/INT	Output from RTC
10 - P_DEBUG	GPIO28	Used by firmware, future other use
11 - SYNC_TIME	GPIO27	Used by firmware
12 - T0_TIME	GPIO26	Used by firmware
13 - SPARE1	GPIO23	Connected to PWO via J2 or future other use
14 - IR LED	GPIO29	IR_LED output before R11
Battery Backup		
BAT+	RTC battery backup	
BAT-	RTC battery backup GND	

17. TULIP4041 Hardware: Development Board

The initial hardware for TULIP4041 is the TULIP-DevBoard, a development board for a commercial Pico module. On this board all signals are available on pins for testing and measurement. A separate module connector board is used for the physical interface to the HP41. There are some minor functional differences between the DevBoard and the module version. The DevBoard is intended to be assembled by the user. The DevBoard is designed for the original Raspberry Pico2 board. Please do not use the Pico-W board. Any other 100% pin-compatible alternative to the Pico (Pimorini for example) should work just fine but may require recompilation of the firmware to adjust for additional features (such as extra memory).

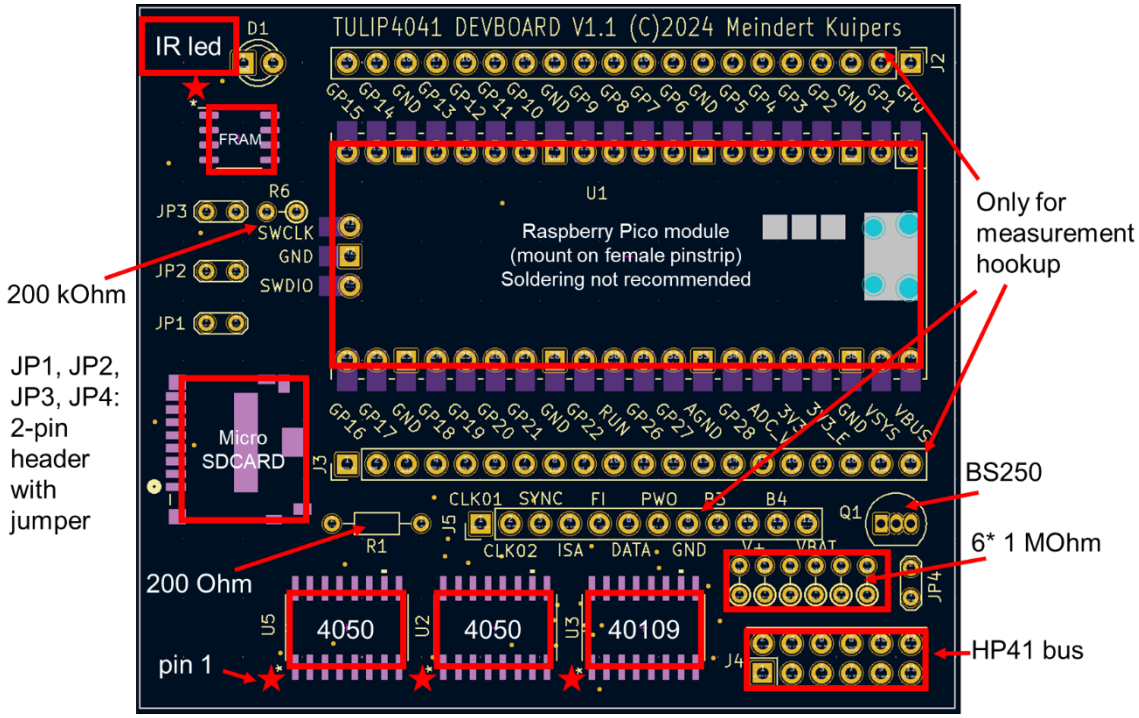
It is very well possible to solder the Pico2 board on the DevBoard, but this is not recommended. It is very hard to unsolder in case of a hardware failure or possible upgrade to another model. When ordering the female headers for the Pico board, ensure that the receptacles are large enough for the Pico pins. Typically turned receptacles may be too small.

The complete Bill Of Material (BOM) is available on the TULIP github pages.

Please follow the steps below in the given order. Do not connect with your precious calculator until instructed.

17.1. Assembling the DevBoard

The DevBoard requires some precise soldering. The figure below shows the position and orientation of the components, pin one is indicated with a red *. Best is to start with the smd components. The headers for probe or analyzer attachment are optional depending on your own preferences. Some of the connections here may be used for future expansions.



U5 is optional. This is a 4050 input level shifter and used only for the FI input tracing. If you do not want to trace FI this component can be left out, but it is highly recommended to keep this in.

To solder an SMD component, the way I do it (using a magnifier lamp):

- Apply a bit of solder on one of the edge pads
- Apply some flux on all pads
- Take the component and verify the value and orientation of pin 1
- Use tweezers to position the component, and solder the edge pad where you applied a bit of solder earlier. Correct the position where needed, such that all pins align with the respective pads
- Solder the pin on the opposite edge, verify the positioning and then solder all other pins
- Check the soldering of all pins with the magnifier

The IR led should be soldered with plenty of pin length above the PCB (about 10mm) and bent in a 90 degree angle away from the board.

Q1 is optional. Mount this if you want the DevBoard to be powered from the HP41. When USB is connected the DevBoard is always powered from USB.

After soldering all components take some time to visually inspect all solder joints and verify the correct orientation before applying power.

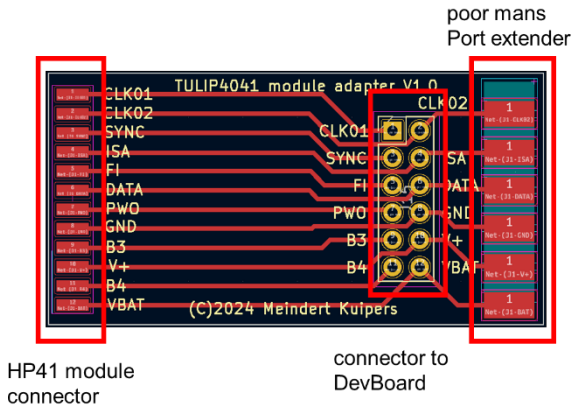
Devboard connections:

J1	micro SD card holder
J2/J3	2* 20 pin header (optional) connections to all Pico pins for analyzer hookup or additional (future) expansions.
J4	2* 6 pin header, connection to the HP41 connector board (use a 12-pin flatcable or solder wires). Do not exaggerate the length of the cable, typically 20 cm will work just fine
J5	12 pin header (optional) connections to all HP41 signals for analyzer hookup
JP1	jumper to enable IR led output. When closed JP3 must be open and correct firmware settings must be observed
JP2	jumper to enable FI tracing (when U5 is mounted)
JP3	jumper to enable PWO output. When closed JP1 must be open and correct firmware settings must be observed. **MUST BE OPEN WHEN USING THE HP82143 PRINTER EMULATION**
JP4	jumper to enable powering the DevBoard from the HP41 BAT line. Only works if Q1 is fitted. Please re-read the section on power consumption
R2..R8	6* 1 MOhm weak pulldowns for stabilizing the HP41 input signals when no HP41 is connected
R1	is the current limiting resistor for the IR led. A value of 200-400 Ohm is just fine and will give you sufficient range to reach your printer or IR receiver at a distance up to 50 cm. Use a low value for more range, but this consumes more power and may draw too much current from the RP2350 I/O pin. The limit is around 12 mA for a single pin

HP41 ConnectorBoard connections:

J1 2* 6 pin header, connection to the HP41 connector board (use a 12-pin flatcable or solder wires). The PCB has pads on both sides of the board and can be soldered on top of the connector as well, although this is not recommended.

J2 HP41 module connector. Please be careful when positioning the connector to ensure proper connection with the HP41. Best is to place the PCB in bottom module case and then position and solder the connector. Keep the PCB with soldered connector in a module case to ensure proper alignment when plugging in the HP41 port.



The connector board has pads on the other end of the PCB to plug a physical module as a kind of poor mans port extender. These are not plated and not suited for a large number of cycles.

Connection between the ConnectorBoard and DevBoard can be done with individual soldered or plugged cables, or a flatcable with IDC headers. Cable length should be around 20 cm, longer may work but do not exaggerate.

17.2. Prepare the micro SD card

The BETA firmware version of the TULIP4041 does not use the SD card, but it will be in a next version of the firmware. There are functions to test the functionality and soldering of the SD card which are necessary.

Prepare a micro SD card formatted with the FAT file system on your host computer. Any size larger than 2 GByte will normally work fine. Create the following subdirectories in the root directory of the micro SD card (although it is not relevant yet)

- ROM, and put some .ROM files in here
- MOD, and addd some .MOD files
- RAW, with some .RAW files
- And put some files in the root directory

Remove (in a safe way) the micro SD card from the host computer and plug it in the slot of the DevBoard.

17.3. Getting the DevBoard up and running

After visual inspection remove any jumpers, these will be used later. Do NOT connect with your calculator yet. First carefully seat the Pico2 board and verify if it is properly connected.

You may now connect the Pico2 board with your host computer using a USB cable. If the Pico2 board is new it does not have any firmware and will go straight into BOOTSEL mode. Otherwise disconnect from USB and put it into BOOTSEL mode by pushing the button on the Pico board while plugging the USB cable. It is also possible to use Picotool to enter BOOTSEL mode, please refer to the Pico documentation for more information.

With the USB cable plugged it makes sense to watch out for smoke or overheated components, although that is unlikely, but may depend on your soldering skills. If you have one it might be useful to use a USB power meter to check for excessive power consumption. Expected current should be under 20-30 mA.

Prepare your host computer with a terminal emulator and have the firmware file (with .uf2 extension) at hand. Keep the Pico connected with USB to your host PC. Do not connect your calculator yet.

Once the TULIP4041 firmware is loaded it is not possible to use Picotool for this. When the Pico is in BOOTSEL mode a USB disk drive is exposed to your host PC. Simply copy the firmware file (with extension .uf2) to this drive. The Pico will reset and start to run the TULIP4041 firmware. Your host PC may respond (if enabled) with sounds that new USB devices are connected.

Now verify if your system registers 5 new COM ports and a USB disk drive. The system may complain about a USB drive not being ready, that is fine.

Start the terminal emulator and connect with the TULIP4041 Command Line Interface (CLI). This is available on one of the new COM or ttyACM ports. Typically the CLI is on the lowest numbered (new) port, but this may differ between systems. The baud rate does not matter. When connected with the correct port the CLI will respond with a welcome message. Try multiple ports to find the CLI, and verify in the system settings of the host PC if all 5 COM ports are visible. When the CLI first connects it will immediately show the welcome message and prompt. You are now ready for the next steps to test the DevBoard and make it operational. The tests are intended to verify the soldering joints.

- ***Initial situation***

  The TULIP is connected with USB and the latest TULIP4041 firmware is loaded. The TULIP welcome message is shown in the terminal emulator connected to the CLI virtual serial port. The HP41 is NOT connected and NO jumpers are placed. Have a multimeter at hand. Your HP41 is near the test setup and has no modules plugged. The calculator may be an HP41C, -CV or -CX. It is possible to use an HP41CL of it is set to the lowest (original) speed an no (virtual) ROMs are plugged.

- ***Find the USB Serial Ports***

  To verify the COM port numbers of all serial ports open an instance of your favorite terminal emulator and connect to each USB virtual serial port you find. Do not worry about the baud rate. In the terminal with the CLI (the one with the TULIP welcome message) type the command: ***system cdcident***. This will send an identification string to each port. Best make a note of the COM port number and function for later use. For the later tests it is best to close the instance of the terminal emulator connected to the HP-IL virtual serial port.

- ***SD card test***

  Put the SD card in the slot on the DevBoard and type ***sdcard status*** in the CLI. This will give the SD characteristics and mount the card. Then type ***dir*** to get a listing of the files in the root directory, and ***dir rom*** to list the files in the ROM subdirectory.

Type ***sdcard connect*** to expose the uSD card filesystem to the host computer, and check if that is indeed the case (the sdcard should be connected to the host PC automatically if a card was plugged during startup).

- ***FRAM test***

  FRAM is tested with the functions for Extended Memory. No plugging of a virtual module is needed. The emulated Extended Memory will be erased with this operation! First type the command ***fram dump*** in the CLI. This will produce a listing of the FRAM contents and should show all zeroes if it wasa brand new FRAM IC. Then type ***fram INIT*** to initialize the FRAM file system, and type ***fram dump 0*** and scroll back to the first lines, it should show the initialized FRAM file system:

```
TULIP> fram dump 0
 dumping FRAM contents from address 0x00000 [next: 0x01000]
00000  41 54 55 4C 49 50 34 30 34 31 20 46 52 41 4D 20   ATULIP4041 FRAM
00010  48 45 41 44 45 52 00 43 E8 03 00 00 D4 A1 00 20   HEADER.C.......
00020  1F 00 00 00 00 01 00 00 30 2E 39 37 20 42 45 54   ........0.97 BET
00030  41 00 00 20 A0 C8 00 20 4E 55 4B 45 49 4E 49 00   A.. ... NUKEINI.
00040  01 00 00 00 0F A5 00 00 00 00 00 00 00 00 00 00   ................
00050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
00060  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
00070  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
00080  41 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00   A@..............
00090  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
000A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
000B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
000C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
000D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
000E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
000F0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
00100  30 54 55 4C 49 50 5F 43 6F 6E 66 69 67 2E 54 43   0TULIP_Config.TC
00110  46 00 41 44 45 52 00 43 E8 03 00 00 D4 A1 00 20   F.ADER.C.......
00120  02 02 00 00 2A 03 00 00 00 00 00 00 00 00 00 00   ....*...........
00130  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
00140  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
00150  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
00160  00 00 00 00 01 00 01 00 00 00 00 00 00 00 01 00   ................
00170  00 00 00 00 00 00 00 00 01 00 01 00 10 27 20 00   .............' .
00180  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
00190  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
001A0  00 00 04 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
```

- ***Configuration test***

  FRAM contains the persistent configuration settings and these should now be initialized. Type the command ***system configlist***. Item #92 must contain the value 4041 (Global settings initialized). The other values are the default settings

This concludes the first part of the test. In case of problems do the following:

- Visually inspect the DevBoard if all components are placed correctly, on the right place and with the correct orientation. Pin 1 is indicated with a * on the PCB silkscreen, and with a dot on the component itself
- Visually inspect all soldering joints (with magnifiers) of the FRAM and micro SD card holder for good soldering contacts and make certain that there are no shorts between component pins
- Check if the Pico board is properly seated and if all solder joints of the Pico connector are good. The TULIP firmware will run even when it is not plugged on the DevBoard.

You may now connect your calculator. Unpower the DevBoard (disconnect USB) and remove all physical modules from the calculator. Connect the DevBoard and ConnectorBoard with a 1-1 cable or individual wires. The B3 and B4 signals are not used and it does not matter which port is used. The calculator should have its own power source. Put the ConnectorBoard in an empty module shell for proper alignment with the HP41 module port.

Best practice is to first connect the DevBoard with USB, and then insert the module in the calculator.

- ***Power test and initial connection***

  Apart from the terminal emulator with the CLI also start an instance of the terminal emulator and connect it with the tracer virtual serial port (verify that with the ***system cdcident*** command). The CLI will give a message when the tracer port is connected.

  Switch your calculator on and verify if it is still working correctly with the TULIP4041 connected. When the HP41 is running the LED on the Pico board will be briefly on. The window with the tracer connected should now show the activity from the calculator. Tracing should be enabled by default, otherwise enable the tracer with the ***tracer trace*** command. Pressing a key in the tracer windows also toggles the operation of the tracer.

  Switch the calculator off.

  Use a multimeter to verify the voltages on the HP41 BAT and V+ on the pins or pads at J5. These should both be around 6V, when the calculator is off V+ will be a bit lower.

- ***HP41 interface test***

  Switch the calculator on again and verify that there is activity in the tracer. This means that the input to the TULIP4041 from the HP41 is working. With the calculator off you can now close the jumper JP2, but only if you have U5 (the additional 4050 level shifter) mounted. This will allow you to trace the FI signal. If your U5 is not mounted then leave JP2 open. You will only see activity on the FI line when a relevant peripheral is connected, the FI signal should normally show only dashes.

  In the CLI type the command ***plug hpil***. This will plug the HP-IL module (but not yet the HP-IL printer ROM). Now switch the calculator on. In the trace windows you should see activity, and occasionally the FI lines in use. Do a CAT 2 on the calculator to verify that the HP-IL module is now plugged.

  For the next tests the instance of the terminal emulator connected to the HP-IL virtual serial port must be closed. The firmware closes the virtual HP-IL loop when nothing is connected to that port. When something is connected the loop is not closed and a TRANSMIT ERR will result. Unless of course a virtual HP-IL device is connected on your host, like ILPer or pyILPer.

  In the HP41 you can now key in the DIR command. This should return with the message NO DRIVE. If this does not happen first do a reboot of the TULIP system by typing the command ***system REBOOT***, and try again. In this case you can skip plugging the HP-IL module, as this will remain plugged.

  When successful you have now verified the connections of the level shifters. There are now 2 more tests for the connections.

  First we will test the ISA power on feature. Make certain that your calculator is not running. It may be powered off or in standby mode. Also have the tracer window standby and keep an eye on it. In the CLI give the command ***system poweron***, and you should see activity in the tracer (but no activity on the calculator). Be aware that this will not switch on the calculator when it was off!

Now close jumper JP3 and open jumper JP1. **Ensure that the printer ROM is NOT plugged!** We will now drive PWO low while the calculator is running, and this act like a hard emergency stop for the HP41. Enter a program with an endless loop and start it, or start a CAT 3 on the HP41. While the calculator is running give the CLI command ***system calcreset***, and verify that the calculators halts and the tracer also stops (after the trace buffer is emptied). After this test open JP3 and close JP1 to prepare for the next and last test.

- ***IR Led test***

  Final test is for the infrared LED. For this you will need an IR to USB serial receiver and align with the IR LED on the DevBoard. Connect this to your host computer and start the HP82240 simulator in HP82143 mode (Windows only). On a no-Windows system best is to connect a terminal emulator. **Ensure that jumper 1 is closed and jumper 3 is open. Never plug the printer with jumper 3 closed!**

  The TULIP IR printing currently does not work with a real HP82240 printer.

  In the CLI type ***plug printer*** to plug the HP82143 ROM image and enable the emulation. Then do the following:

  - Type ***printer***, verify the status
  - Type ***printer power*** if the printer was powered off
  - Type ***printer paper*** if it was out of paper
  - On the calculator key CAT 2 to verify of the printer ROM is plugged
  - In the TULIP CLI key ***printer trace*** and run a CAT 2 again. You should see the catalog listing in the printer simulator, or a catalog listing with some control characters in a terminal emulator
  - If you do not have the IR receiver, start a long printing operation, like PRP PRPLOT. Point your mobile phone camera straight into the IR LED and you should see a faint blue glow that stops when the printing is done
  - From version 0.91 the CLI has a command that tests the IR LED by sending a string to the LED, to use this just type ***printer irtest***, no need to plug the printer ROM.

You have now tested all functions and soldering of the DevBoard. Congratulations!

18. References

All firmware files, documentation and schematics are available at my GitHub pages: https://github.com/mjakuipers/TULIP-DevBoard.

Several videos about my calculator activities, including the TULIP, are available on my YouTube channel: https://www.youtube.com/@TheMeinable

Reference	URL	Description
HP Museum Forum	https://www.hpmuseum.org/forum/index.php	The best place to get help and general information
V41 emulator	HP41.org and https://hp.giesselink.com/v41.htm	HP41 emulator for Windows
HP-IL emulation	https://hp.giesselink.com/hpil.htm	Emulated hardware on your PC, to use with the PIL Box or the V41 emulator
PIL Box	http://www.jeffcalc.hp41.eu/hpil/index.html	HP-IL to PC USB based link
EMU41	http://www.jeffcalc.hp41.eu/hpil/index.html	HP41 emulator, DOS based
HP41.org	www.hp41.org	The best source for manuals, books, ROM images
Clonix, NoVRAM and HP82143 emulator	https://www.clonix41.org/	Hardware plug-in configurable modules, and a printer emulator
41CL Homepage	http://systemyde.com/hp41/index.html	Home page for the 41CL Calculator and MAXX module
SDK41	www.hp41.org	Old-school mcode toolchain for DOS
41CL Other Docs	http://systemyde.com/hp41/documents.html	Includes the OSX3 manual
41CL File area	http://systemyde.com/hp41/files.html	Download links of the current ROM repository and various tools, which includes the CL Updater and clreader/clwriter plus some other goodies
HP-IL virtual devices	https://github.com/bug400/pyilper	pyILPer, a python based HP-IL emulator with printer and mass storage emulation
Calypsi	https://www.calypsi.cc/	Modern mcode and RPN toolchain with debugger for Linux, MacOS and Windows
Raspberry Pi RP2350 SDK	https://www.raspberrypi.com/documentation/pico-sdk/index_doxygen.html	All info on the SDK

Raspberry Pi RP2350 chip	https://www.raspberrypi.com/products/rp2350/	All info on the RP2350 chip
Raspberry Pi Pico2 module	https://www.raspberrypi.com/products/raspberry-pi-pico-2/	All info on the Pico2 module
PCF8523 RTC	https://www.nxp.com/part/PCF8523T#/	Datasheet of the RTC chip
KiCad	https://www.kicad.org/	Website of KiCad, used for the design of the TULIP PCB's
HP41UC by Leo Duran	https://sourceforge.net/projects/hp41uc/ File Converter (LIF. P41, BIN, RAW. etc), Compiler (text to binary), De-compiler (binary to text), Barcode Generator (PCL3, Postscript images)	Program to convert HP41 user code between various formats including generating barcode
Book	HP41CX Programmers Handbook by Poul Kaarup	Available from hp41.org. A must have for mcode programmers
RedEye IR Receiver	https://github.com/mjakuipers/HP-RedEye-Receiver, based on https://www.mh-aerotools.de/hp/red-eye/index.htm	Infrared receiver for a PC-based Printer Simulator

19. Change log

Version	date	description
00.01.01	June 2024	Initial version for beta release
00.01.02	September 2024	Edited and expanded for first public release
00.01.03	September 2024	Preparing for change to RP2350/Pico2 and first public PCB's
00.01.04	October 2024	Migrated to RP2350/Pico2, firmware updates documented
00.02.01	March 2025	Update to firmware specifications
00.03.01	June 2025	Update to firmware 0.9 BETA 1 and preliminary Module version
00.03.02	August 2025	Update to firmware 0.92 BETA 2 Added references, general edits
00.04.02	December 2025	Update to firmware 0.96, split user manual from background Added references, general edits wand and w command added
00.04.03	Februari 2026	Added test procedure after connector soldering
00.04.04	March 2026	Updated to firmware 0.97 (FRAM file system, qrom and hepram)
00.04.05	June 2026	Update to firmware 0.990
00.04.06	Oktober 2026	Update to V1.1 hardware

